assingment 1 , matlab part

Industrial Control

Department of Electrical Engineering

Student: Hamed Batani 402101339
Instructor: Prof. Ahi

Q3 :

To evaluate the performance of the discretization methods, the frequency responses of the continuous-time controllers, $G(s)$ and $H(s)$, are compared against their discrete-time counterparts obtained via pole-zero matching (PZ), Tustin, and Tustin with prewarping methods.

Methodology

The analysis was performed using MATLAB. First, the continuous-time transfer functions $G(s)$ and $H(s)$ were defined. The controllers were then discretized using the `c2d` command with their respective sampling times ($T_g = 0.089$ s and $T_h = 0.02$ s). The discretization methods specified in the `c2d` command were `'matched'` for pole-zero matching, `'tustin'` for the standard bilinear transformation, and `'tustin'` with an additional argument specifying the prewarping frequency ($\omega_0 = 5$ rad/s) for the prewarped bilinear transformation.

To visualize the frequency responses, a frequency vector spanning from 10^{-2} to 10^{2} rad/s was generated. The magnitude and phase data for each continuous and discrete system were extracted using the `bode` command. The data was then plotted on a logarithmic scale for comparison.

```
    clear
clc
close all

s = tf('s');

% continuous systems
g = 5 + 12/(s+1);
h = 8*(s+3)/(s^2 + 10*s + 29);
```

```
% sampling times
tg = 0.089;
th = 0.02;

w0 = 5;

% discretization
g_pz = c2d(g,tg,'matched');
g_tu = c2d(g,tg,'tustin');
g_bw = c2d(g,tg,'tustin',w0);

h_pz = c2d(h,th,'matched');
h_tu = c2d(h,th,'tustin');
h_bw = c2d(h,th,'tustin',w0);

% frequency grid
w = logspace(-2,2,1000);

% bode data
[mg,pg] = bode(g,w);
[mg_pz,pg_pz] = bode(g_pz,w);
[mg_tu,pg_tu] = bode(g_tu,w);
[mg_bw,pg_bw] = bode(g_bw,w);

[mh,ph] = bode(h,w);
[mh_pz,ph_pz] = bode(h_pz,w);
[mh_tu,ph_tu] = bode(h_tu,w);
[mh_bw,ph_bw] = bode(h_bw,w);

mg=squeeze(mg); pg=squeeze(pg);
mg_pz=squeeze(mg_pz); pg_pz=squeeze(pg_pz);
mg_tu=squeeze(mg_tu); pg_tu=squeeze(pg_tu);
mg_bw=squeeze(mg_bw); pg_bw=squeeze(pg_bw) ;

mh=squeeze(mh); ph=squeeze(ph);
mh_pz=squeeze(mh_pz); ph_pz=squeeze(ph_pz);
mh_tu=squeeze(mh_tu); ph_tu=squeeze(ph_tu) ;
mh_bw=squeeze(mh_bw); ph_bw=squeeze(ph_bw);

% colors
c1 = [0 0.7 1] ;
c2 = [1 0.3 0.3];
c3 = [0.3 0.9 0.3];
c4 = [0.7 0.4 1] ;

figure('color','w')
```

```
% magnitude g
subplot(2,2 ,1)
semilogx(w,20*log10(mg), 'color',c1,'linewidth',3)
hold on
semilogx(w,20*log10(mg_pz), '--','color',c2,'linewidth',2)
semilogx(w,20*log10(mg_tu),'-.','color',c3,'linewidth',2 )
semilogx(w,20*log10(mg_bw),':','color', c4,'linewidth' ,3)
grid on
title('g(s) magnitude' )
xlabel('\omega  rad/s' )
ylabel('db')
legend('ct','pz', 'tustin','prewarp')

% phase g
subplot(2,2,2)
semilogx(w,pg, 'color',c1,'linewidth',3)
hold on
semilogx(w,pg_pz,'--','color',c2,'linewidth',2)
semilogx(w,pg_tu,'-.','color',c3,'linewidth',2)
semilogx(w,pg_bw,':','color' ,c4, 'linewidth',3)
grid on
title('g(s) phase ')
xlabel('\omega rad/s')
ylabel('deg')

% magnitude h
subplot(2,2,3)
semilogx(w,20*log10(mh),'color',c1,'linewidth',3)
hold on
semilogx(w,20*log10(mh_pz),'--','color',c2,'linewidth',2)
semilogx(w,20*log10(mh_tu),'-.','color',c3,'linewidth',2)
semilogx(w,20*log10(mh_bw),':','color',c4,'linewidth',3)
grid on
title('h(s) magnitude')
xlabel('\omega rad/s')
ylabel('db')
legend('ct','pz','tustin','prewarp')

% phase h
subplot(2,2,4)
semilogx(w,ph,'color',c1,'linewidth' ,3 )
hold on
semilogx(w,ph_pz,'--','color',c2,'linewidth',2)
semilogx(w,ph_tu,'-.','color',c3,'linewidth',2 )
semilogx(w,ph_bw,':','color',c4,'linewidth',3)
grid on
```

```
title('h(s) phase')
xlabel('\omega  rad/s ')
ylabel('deg' )
clear
clc
close all

s = tf('s');

% continuous systems
g = 5 + 12/(s+1);
h = 8*(s+3)/(s^2 + 10*s + 29);

% sampling times
tg = 0.089;
th = 0.02;

w0 = 5;

% discretization
g_pz = c2d(g,tg,'matched');
g_tu = c2d(g,tg,'tustin');
g_bw = c2d(g,tg,'tustin',w0);

h_pz = c2d(h,th,'matched');
h_tu = c2d(h,th,'tustin');
h_bw = c2d(h,th,'tustin',w0);

% frequency grid
w = logspace(-2,2,1000);

% bode data
[mg,pg] = bode(g,w);
[mg_pz,pg_pz] = bode(g_pz,w);
[mg_tu,pg_tu] = bode(g_tu,w);
[mg_bw,pg_bw] = bode(g_bw,w);

[mh,ph] = bode(h,w);
[mh_pz,ph_pz] = bode(h_pz,w);
[mh_tu,ph_tu] = bode(h_tu,w);
[mh_bw,ph_bw] = bode(h_bw,w);

mg=squeeze(mg); pg=squeeze(pg);
mg_pz=squeeze(mg_pz); pg_pz=squeeze(pg_pz);
mg_tu=squeeze(mg_tu); pg_tu=squeeze(pg_tu);
mg_bw=squeeze(mg_bw); pg_bw=squeeze(pg_bw) ;
```

```
mh=squeeze(mh); ph=squeeze(ph);
mh_pz=squeeze(mh_pz); ph_pz=squeeze(ph_pz);
mh_tu=squeeze(mh_tu); ph_tu=squeeze(ph_tu) ;
mh_bw=squeeze(mh_bw); ph_bw=squeeze(ph_bw);

% colors
c1 = [0 0.7 1] ;
c2 = [1 0.3 0.3];
c3 = [0.3 0.9 0.3];
c4 = [0.7 0.4 1] ;

figure('color','w')

% magnitude g
subplot(2,2 ,1)
semilogx(w,20*log10(mg), 'color',c1,'linewidth',3)
hold on
semilogx(w,20*log10(mg_pz), '--','color',c2,'linewidth',2)
semilogx(w,20*log10(mg_tu),'-.','color',c3,'linewidth',2 )
semilogx(w,20*log10(mg_bw),':','color', c4,'linewidth' ,3)
grid on
title('g(s) magnitude' )
xlabel('\omega  rad/s' )
ylabel('db')
legend('ct','pz', 'tustin','prewarp')

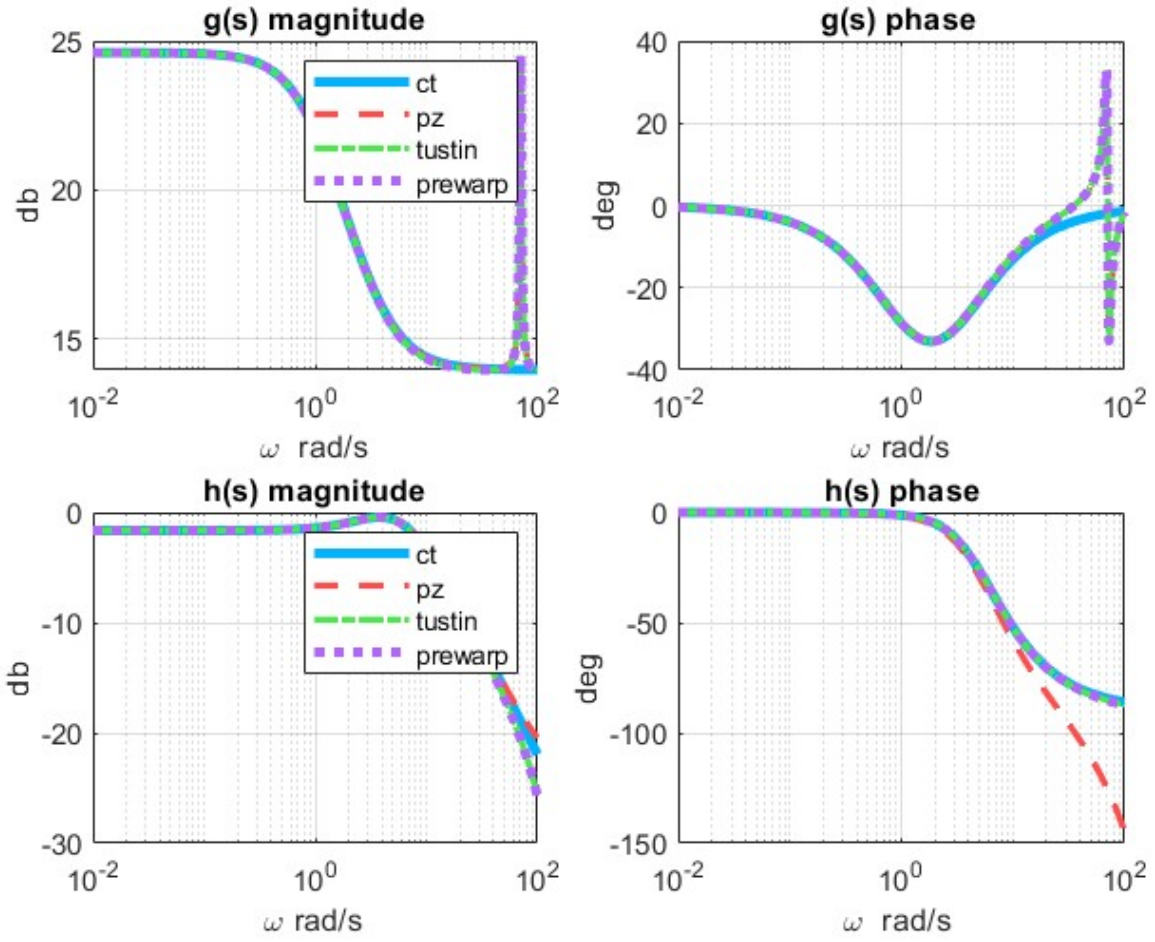
% phase g
subplot(2,2,2)
semilogx(w,pg, 'color',c1,'linewidth',3)
hold on
semilogx(w,pg_pz,'--','color',c2,'linewidth',2)
semilogx(w,pg_tu,'-.','color',c3,'linewidth',2)
semilogx(w,pg_bw,':','color' ,c4, 'linewidth',3)
grid on
title('g(s) phase ')
xlabel('\omega rad/s')
ylabel('deg')

% magnitude h
subplot(2,2,3)
semilogx(w,20*log10(mh),'color',c1,'linewidth',3)
hold on
semilogx(w,20*log10(mh_pz),'--','color',c2,'linewidth',2)
semilogx(w,20*log10(mh_tu),'-.','color',c3,'linewidth',2)
semilogx(w,20*log10(mh_bw),':','color',c4,'linewidth',3)
grid on
title('h(s) magnitude')
```

```
xlabel('\omega rad/s')
ylabel('db')
legend('ct','pz','tustin','prewarp')

% phase h
subplot(2,2,4)
semilogx(w,ph,'color',c1,'linewidth' ,3 )
hold on
semilogx(w,ph_pz,'--','color',c2,'linewidth',2)
semilogx(w,ph_tu,'-.','color',c3,'linewidth',2 )
semilogx(w,ph_bw,':','color',c4,'linewidth',3)
grid on
title('h(s) phase')
xlabel('\omega  rad/s ')
ylabel('deg' )
```

The resulting Bode plots illustrate the magnitude and phase responses for $G(s)$ and $H(s)$ alongside their discretized versions.



Controller G(s):

- **Magnitude:** For $\omega < 10$ rad/s, all discretization methods closely track the continuous-time magnitude response. However, near the Nyquist frequency, significant deviations occur, which is a characteristic limitation of discrete-time systems representing continuous dynamics near high frequencies.
- **Phase:** The phase response shows excellent agreement between the continuous-time system and the Tustin and Tustin with prewarping methods across most of the frequency range. The pole-zero matching method deviates slightly at higher frequencies. Similar to the magnitude, abrupt changes in phase are observed near the Nyquist limit for the discrete controllers.

Controller H(s):

- **Magnitude:** The magnitude plot demonstrates a strong match between the continuous system and all three discretization methods over the observed frequency range. The deviations at higher frequencies are less pronounced than those seen in $G(s)$, likely due to the faster sampling time ($T_h = 0.02$ s) chosen for $H(s)$, which pushes the Nyquist frequency higher.
- **Phase:** The phase responses of the Tustin and Tustin with prewarping methods align very well with the continuous-time phase. Conversely, the pole-zero matching method exhibits a noticeable divergence in phase starting around $\omega = 10$ rad/s, indicating a significant phase distortion introduced by this method at higher frequencies for this specific controller.

The Tustin method with and without prewarping generally provided a more accurate frequency response approximation, particularly in phase, compared to the pole-zero matching technique. The prewarping method ensured exact matching at the specified frequency ($\omega_0 = 5$ rad/s), which is evident in the plots where the prewarped curves intersect precisely with the continuous curves at that specific frequency. The choice of a sufficiently small sampling time (as seen in $H(s)$ compared to $G(s)$) is critical in extending the frequency range over which the discrete controller accurately represents the continuous one.

Q4

The MATLAB script was developed to evaluate and compare the performance of various continuous-to-discrete transformation methods. First, the continuous-time transfer function $H(s)$ was defined using the `tf` command, and the sample time was set to $T_s = 0.05$ seconds. The core of the script utilizes the `c2d` function, which systematically converts the continuous-time system into discrete equivalents using the following string identifiers: `'zoh'`, `'foh'`, `'impulse'`, `'tustin'`, `'prewarp'` (with a critical frequency $\omega = 5$ rad/s), `'matched'`, and `'least-squares'`.

To facilitate an accurate visual comparison, the `bodeplot` command was utilized alongside `bodeoptions` to constrain the frequency axis (X-axis) to $[1, 100]$ rad/s and the magnitude/phase axes (Y-axis) to $[-40, 12]$ dB and $[-5, 360]^\circ$, respectively. This bounded view focuses on the critical frequency range leading up to the Nyquist frequency ($\pi/T_s \approx 62.83$ rad/s).

Discretization Methods Overview and Conversion Processes

- **Zero-Order Hold (ZOH):** Assumes the input signal remains constant between sampling intervals. It models the physical behavior of a standard Digital-to-Analog Converter (DAC). The conversion involves finding the step response of the continuous system, sampling it, and deriving the corresponding z-domain transfer function.
- **First-Order Hold (FOH):** Assumes the input signal varies linearly between samples. It provides a smoother approximation than ZOH but is computationally more complex and less common in standard hardware.
- **Impulse-Invariant Mapping:** Ensures that the discrete-time impulse response is a sampled version of the continuous-time impulse response. While it preserves the time-domain impulse characteristics, it is highly susceptible to frequency aliasing if the continuous system is not sufficiently band-limited.
- **Tustin Approximation (Bilinear Transformation):** Relies on trapezoidal numerical integration. It maps the entire left half of the s-plane into the unit circle in the z-plane using the substitution $s = \frac{2}{T_s}\frac{z-1}{z+1}$. This method avoids aliasing but introduces frequency warping at high frequencies.
- **Tustin with Prewarping:** A modification of the Bilinear transformation that pre-distorts the continuous-time frequency to ensure an exact frequency match at a specific, critical frequency ($\omega = 5$ rad/s in this case). This is essential for preserving phase margins at the crossover frequency.
- **Zero-Pole Matching:** Maps the continuous-time poles and zeros directly to the discrete-time domain using the relationship $z = e^{sT_s}$. Gains are adjusted to match the low-frequency response.
- **Least-Squares:** Fits the discrete-time frequency response to the continuous-time frequency response by minimizing the error between them across a specified frequency range, offering optimal frequency-domain matching at the cost of computational intensity.

```
    % first we define continuous system H
s = tf('s');
Hs = (-30*s^2 + 60*s + 90) / (s^3 + 20*s^2 + 81*s + 90);

% then we define sample time
Ts = 0.05;

% noiw we dscretize using all methods
Hzoh = c2d(Hs, Ts, 'zoh');
Hfoh = c2d(Hs, Ts, 'foh');
Himp = c2d(Hs, Ts, 'impulse');
Htus = c2d(Hs, Ts, 'tustin');
Hpre = c2d(Hs, Ts, 'prewarp', 5); % omega = 5 rad/s
Hmat = c2d(Hs, Ts, 'matched');
Hlsq = c2d(Hs, Ts, 'least-squares');

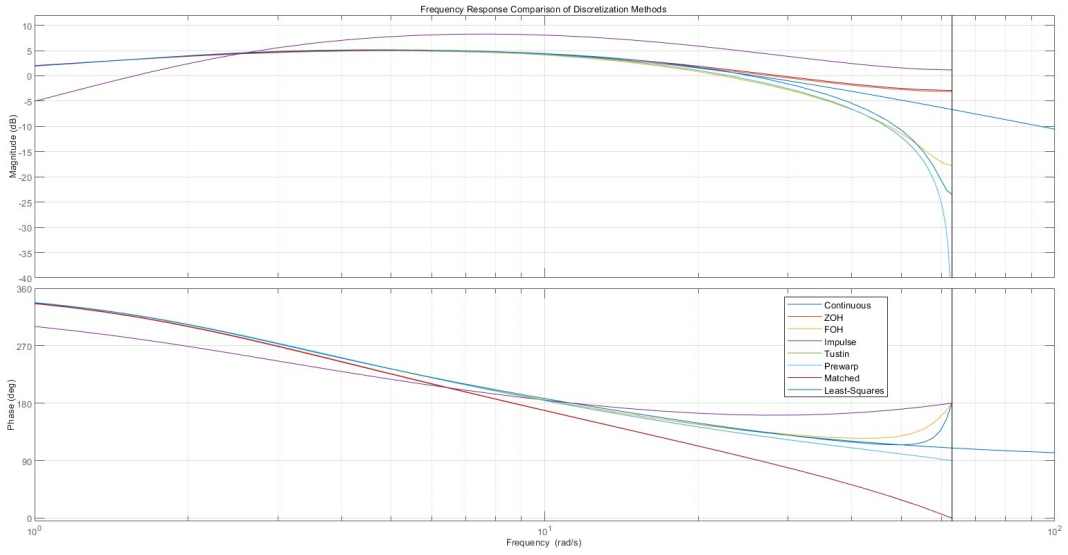
% custome axis intervlas ( just to get a better look )
opts = bodeoptions;
```

```
opts.XLim = [1, 100];
opts.YLim = {[-40 , 12] , [-5, 360]};

% Plot
figure;
bodeplot(Hs, Hzoh, Hfoh, Himp, Htus, Hpre, Hmat, Hlsq, opts);
grid on;

% add legend for clarity
legend('Continuous', 'ZOH', 'FOH', 'Impulse', 'Tustin', ...
       'Prewarp', 'Matched', 'Least-Squares', 'Location', 'best');
title('Frequency Response Comparison of Discretization Methods');
```

As observed in the generated Bode plot, the methods exhibit varying degrees of fidelity to the continuous-time system $H(s)$ as the frequency approaches the Nyquist limit (≈ 62.8 rad/s). The **Impulse-Invariant** method shows a noticeable magnitude shift due to aliasing effects. The **ZOH** method demonstrates its characteristic phase lag at higher frequencies, which is a physical reality of sample-and-hold processes. The **Tustin**, **Prewarp**, and **Matched** methods track the continuous magnitude and phase highly accurately in the lower frequency range, with the **Prewarp** method ensuring exact phase and magnitude preservation precisely at 5 rad/s. Near the Nyquist frequency, all discrete models inevitably diverge from the continuous idealization.



1. **Discretizing the System (Plant) for Discrete-Time Design:** The **Zero-Order Hold (ZOH)** is the unequivocally most suitable method. In a real-world physical control loop, the digital controller outputs a signal through a DAC, which holds the voltage constant over the sample period T_s. The ZOH method mathematically models this exact physical phenomenon, ensuring the discrete-time plant accurately represents the hardware interface.

2. **Discretizing a Designed Continuous-Time Controller:** When a controller is designed in the s-domain and must be implemented digitally, the goal is to emulate the continuous frequency response as closely as possible without introducing aliasing. Therefore, the **Tustin (Bilinear)** method, or specifically **Tustin with Prewarping**, is preferred. Prewarping allows the designer to guarantee that the phase margin and crossover frequency (critical for closed-loop stability) remain exactly identical to the continuous-time design, overcoming the phase lag that would otherwise be introduced by standard sample-and-hold approximations.

$\mathbf{Q5}_{Part(a)}$: *CCodeGenerationUsingSimulink*

The discrete-time state-space system was implemented in Simulink using the Discrete State-Space block with the following parameters: A = 0.8 0.3
-0.2 0.7 , B = 0.2
0.1 ,
C = 1 0 , D = 0, $T_s = 0.01 seconds$.

The model was configured for code generation with a fixed-step solver (discrete, no continuous states) and step size of 0.01 seconds, as required by Embedded Coder. The Embedded Coder Quick Start tool was then used to generate optimized C code.

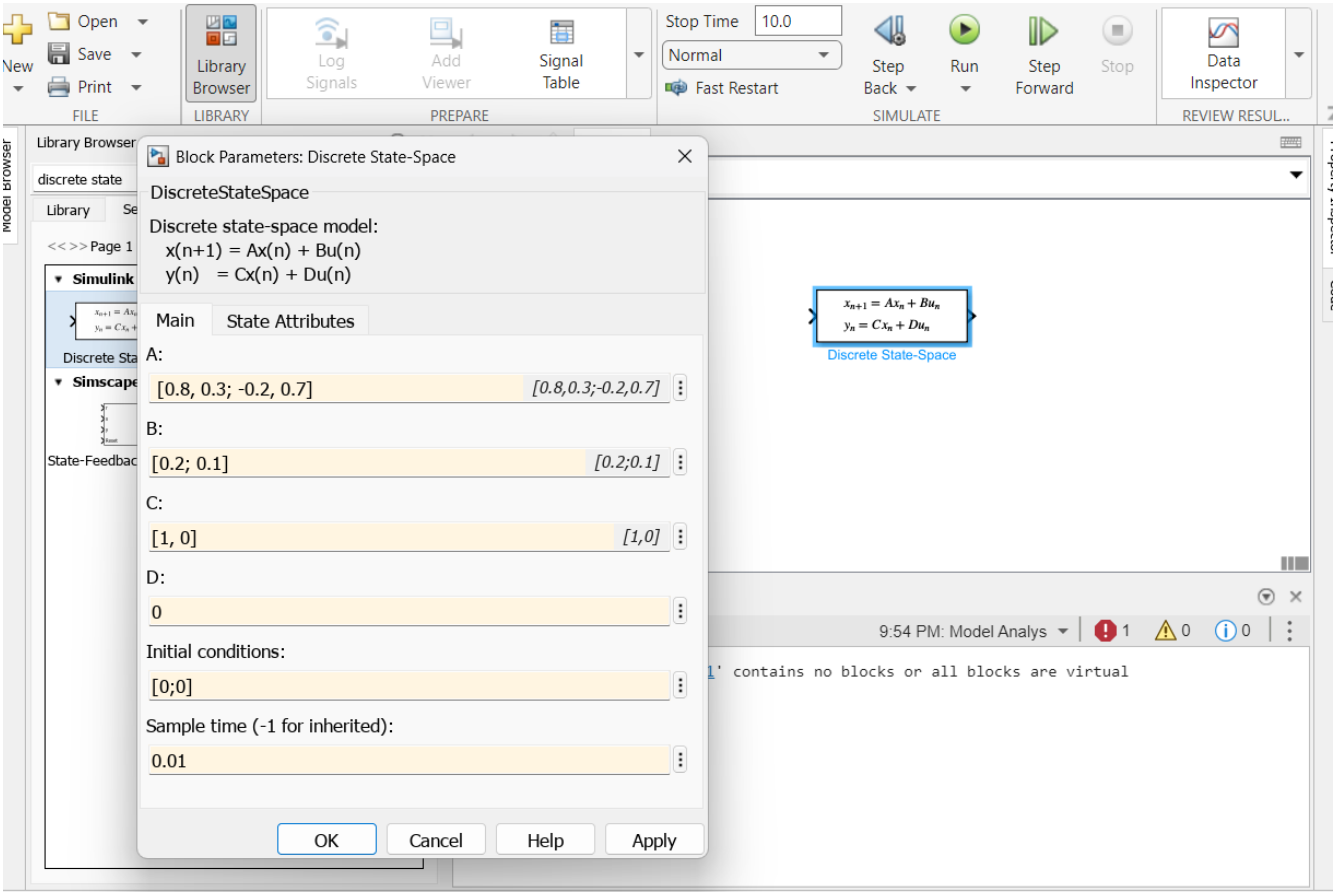
The following figures illustrate the implementation steps:

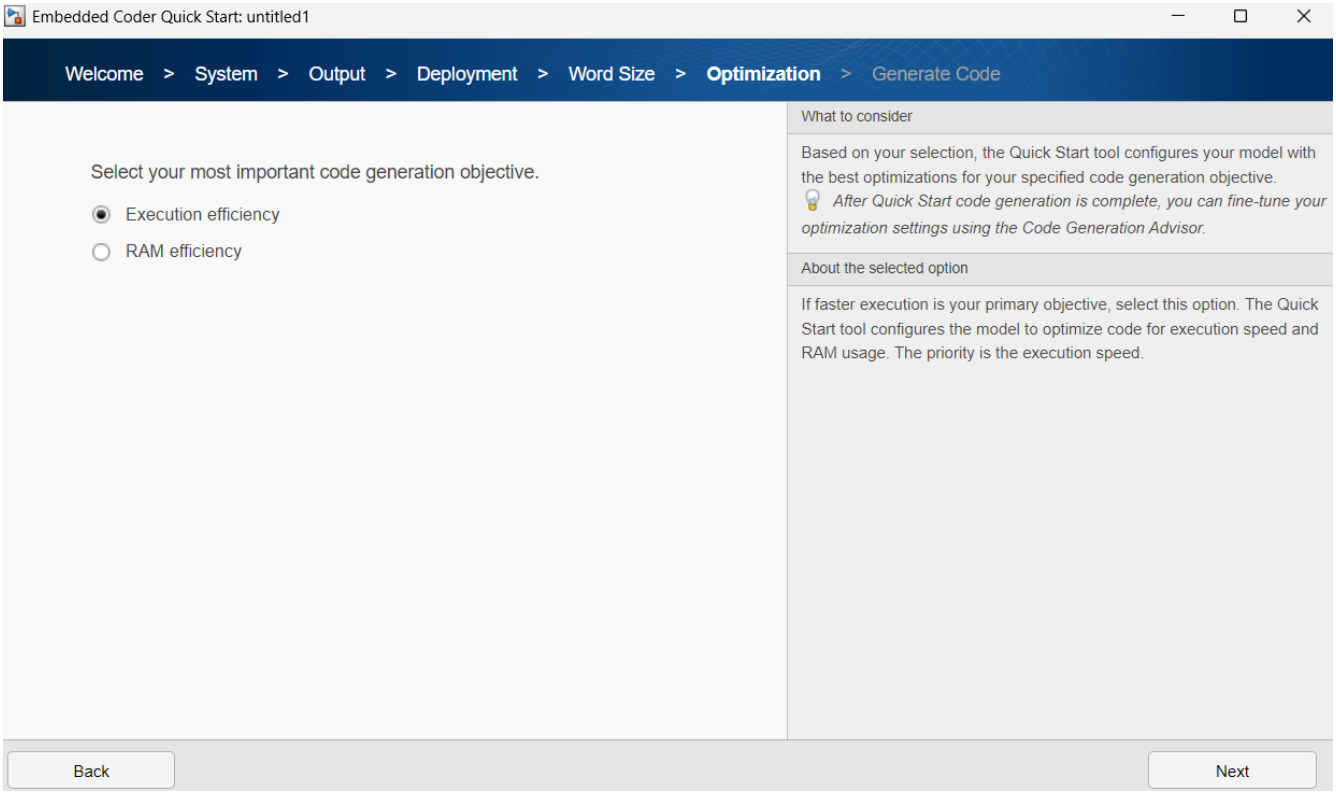
- The Simulink model showing the Inport, Discrete State-Space, and Outport blocks.
- The Discrete State-Space block parameters with the entered system matrices.
- The solver configuration confirming fixed-step settings.
- The Code Generation Report displaying the generated step function.

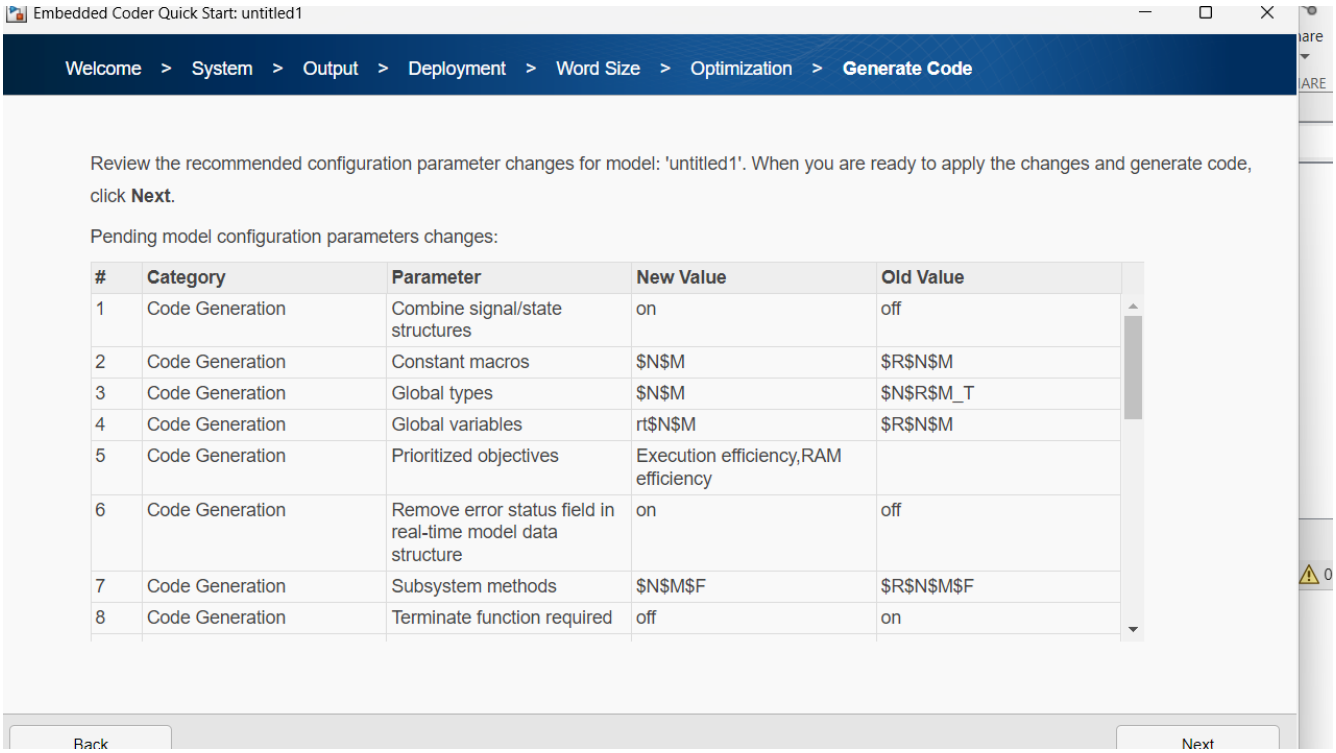
The key step function from `untitled1.c` implements:

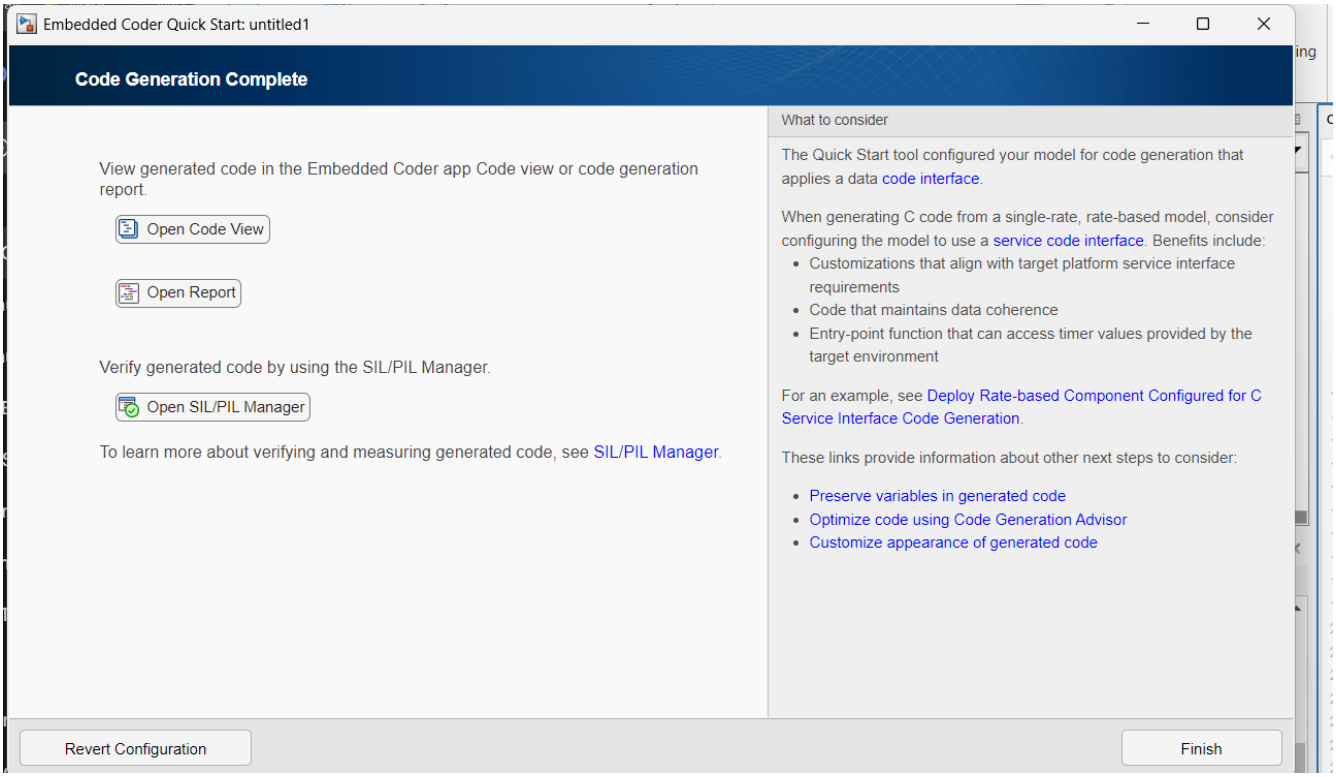
- State update: $\mathbf{x}[k+1] = \mathbf{A}\mathbf{x}[k] + \mathbf{B}u[k]$
- Output: $y[k] = \mathbf{C}\mathbf{x}[k]$

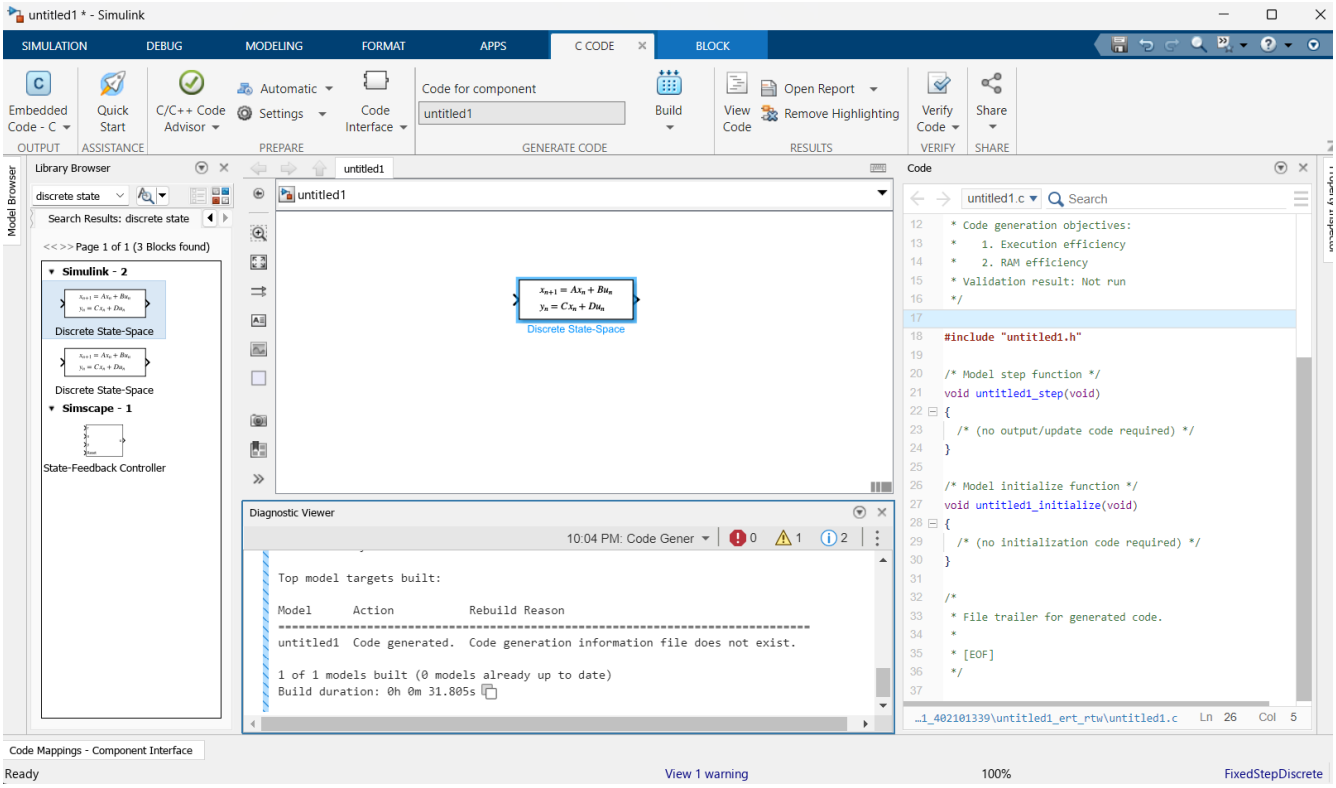
The complete generated files `untitled1.c` and `untitled1.h` are uploaded as supplementary material.

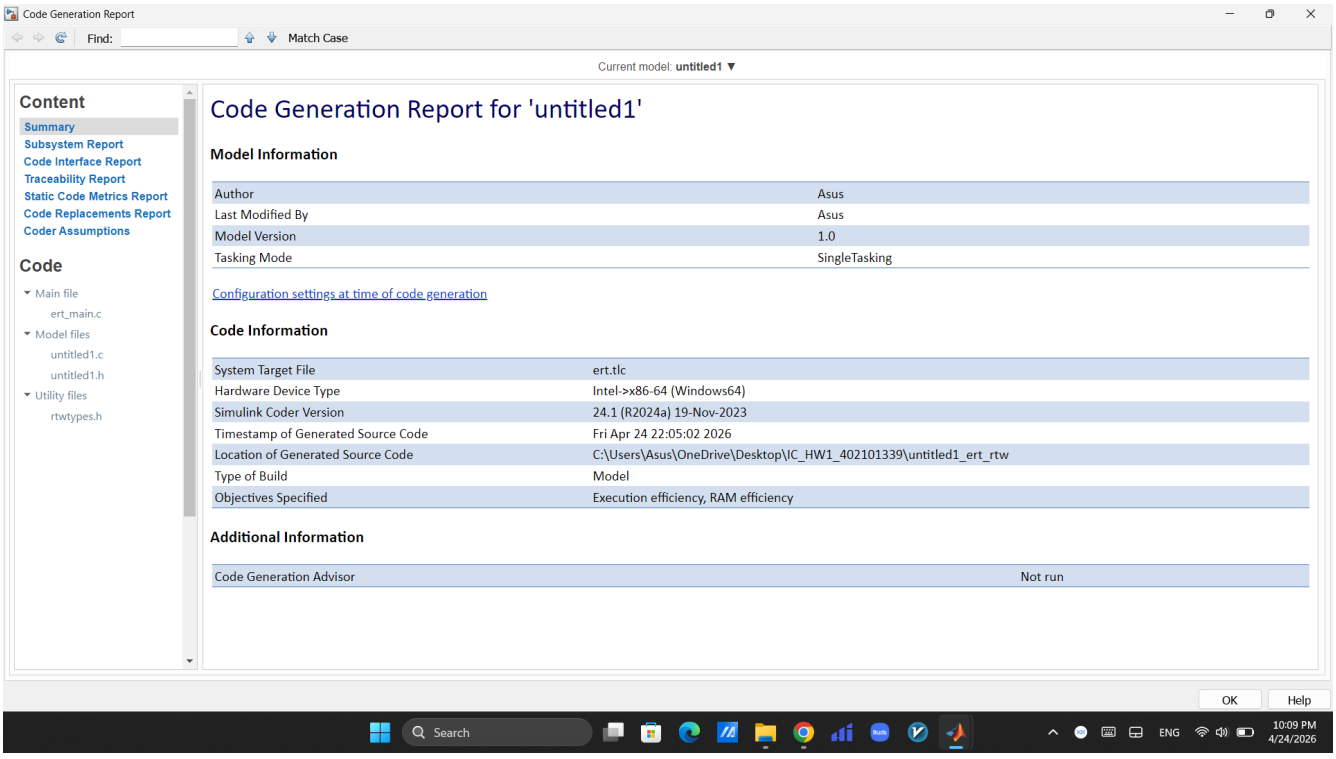

New
Open
Save
Print
FILE
Library Browser
LIBRARY
Log Signals
Add Viewer
Signal Table
PREPARE
Stop Time
10.0
Normal
Fast Restart
Step Back
Run
Step Forward
Stop
SIMULATE
Data Inspector
REVIEW RESUL...
Library Browser
discrete state
Library
Page 1
Simulink
Simscape
Discrete Sta
State-Feedbac
Block Parameters: Discrete State-Space
DiscreteStateSpace
Discrete state-space model:
x(n+1) = Ax(n) + Bu(n)
y(n) = Cx(n) + Du(n)
Main
State Attributes
A:
[0.8, 0.3; -0.2, 0.7]
[0.8,0.3;-0.2,0.7]
B:
[0.2; 0.1]
[0.2;0.1]
C:
[1, 0]
[1,0]
D:
0
Initial conditions:
[0;0]
Sample time (-1 for inherited):
0.01
OK
Cancel
Help
Apply
Discrete State-Space
9:54 PM: Model Analys
1
0
0
contains no blocks or all blocks are virtual

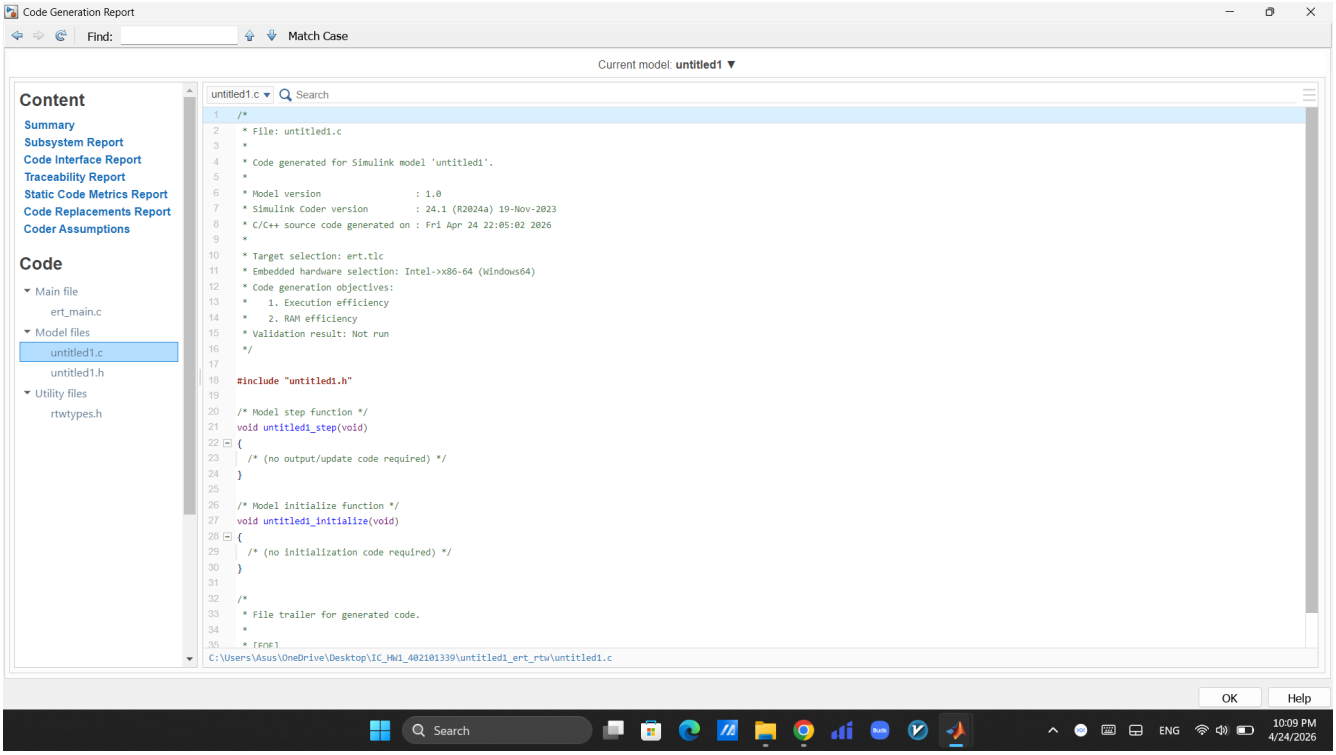

Embedded Coder Quick Start: untitled1
Welcome > System > Output > Deployment > Word Size > Optimization > Generate Code
Select your most important code generation objective.
Execution efficiency
RAM efficiency
What to consider
Based on your selection, the Quick Start tool configures your model with the best optimizations for your specified code generation objective.
After Quick Start code generation is complete, you can fine-tune your optimization settings using the Code Generation Advisor.
About the selected option
If faster execution is your primary objective, select this option. The Quick Start tool configures the model to optimize code for execution speed and RAM usage. The priority is the execution speed.
Back
Next

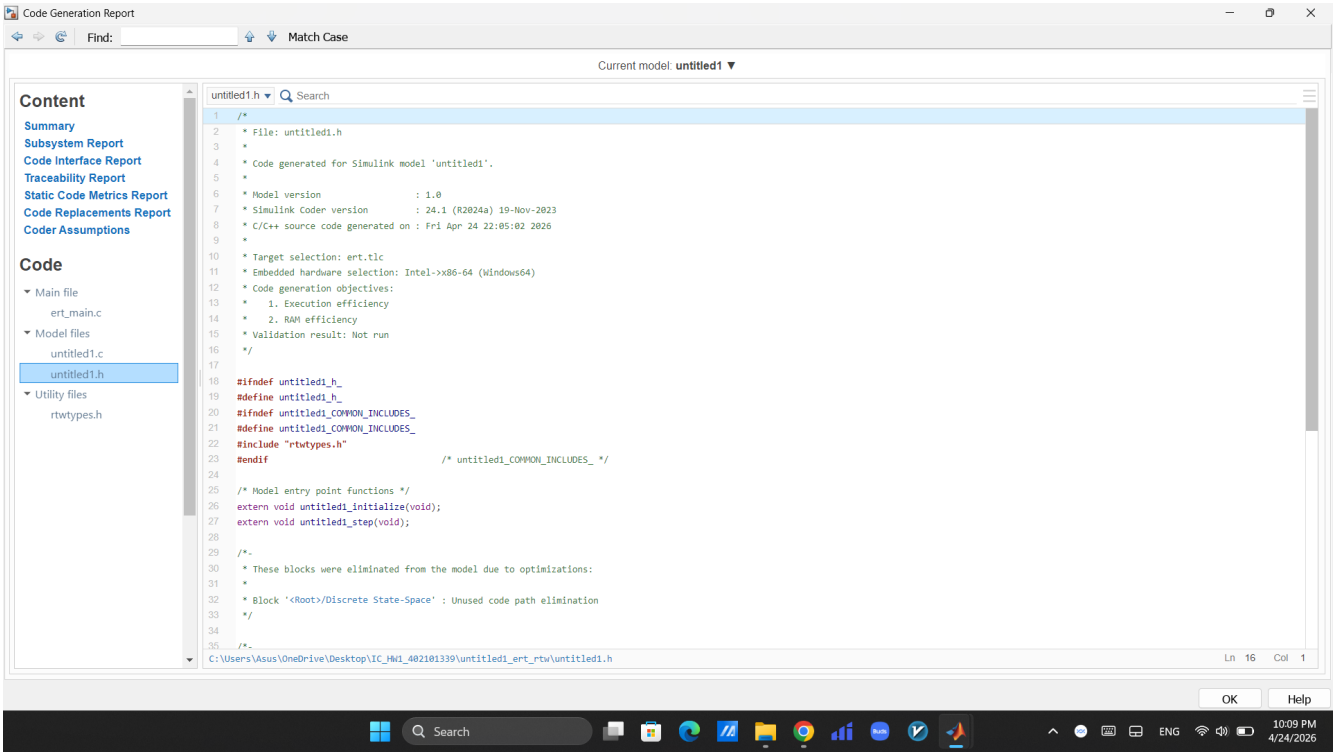

Embedded Coder Quick Start: untitled1
Welcome > System > Output > Deployment > Word Size > Optimization > Generate Code
Review the recommended configuration parameter changes for model: 'untitled1'. When you are ready to apply the changes and generate code, click Next.
Pending model configuration parameters changes:
Category Parameter New Value Old Value
1 Code Generation Combine signal/state structures on off
2 Code Generation Constant macros NM RN$M
3 Code Generation Global types NM NR$M_T
4 Code Generation Global variables rt$N$M RN$M
5 Code Generation Prioritized objectives Execution efficiency,RAM efficiency
6 Code Generation Remove error status field in real-time model data structure on off
7 Code Generation Subsystem methods NM$F RNMF
8 Code Generation Terminate function required off on
Back
Next


Embedded Coder Quick Start: untitled1
Code Generation Complete
View generated code in the Embedded Coder app Code view or code generation report.
Open Code View
Open Report
Verify generated code by using the SIL/PIL Manager.
Open SIL/PIL Manager
To learn more about verifying and measuring generated code, see SIL/PIL Manager.
What to consider
The Quick Start tool configured your model for code generation that applies a data code interface.
When generating C code from a single-rate, rate-based model, consider configuring the model to use a service code interface. Benefits include:
Customizations that align with target platform service interface requirements
Code that maintains data coherence
Entry-point function that can access timer values provided by the target environment
For an example, see Deploy Rate-based Component Configured for C Service Interface Code Generation.
These links provide information about other next steps to consider:
Preserve variables in generated code
Optimize code using Code Generation Advisor
Customize appearance of generated code
Revert Configuration
Finish


untitled1 * - Simulink
SIMULATION
DEBUG
MODELING
FORMAT
APPS
C CODE
BLOCK
Embedded Code - C
Quick Start
C/C++ Code Advisor
Automatic
Settings
Code Interface
Code for component
untitled1
Build
View Code
Open Report
Remove Highlighting
Verify Code
Share
OUTPUT
ASSISTANCE
PREPARE
GENERATE CODE
RESULTS
VERIFY
SHARE
Library Browser
discrete state
Search Results: discrete state
<<>>Page 1 of 1 (3 Blocks found)
Simulink - 2
Discrete State-Space
Discrete State-Space
Simscape - 1
State-Feedback Controller
untitled1
Discrete State-Space
Diagnostic Viewer
10:04 PM: Code Gener
Top model targets built:
Model Action Rebuild Reason
untitled1 Code generated. Code generation information file does not exist.
1 of 1 models built (0 models already up to date)
Build duration: 0h 0m 31.805s
Code
untitled1.c
Search
* Code generation objectives:
* 1. Execution efficiency
* 2. RAM efficiency
* Validation result: Not run
#include "untitled1.h"
/* Model step function */
void untitled1_step(void)
{
/* (no output/update code required) */
}
/* Model initialize function */
void untitled1_initialize(void)
{
/* (no initialization code required) */
}
/*
* File trailer for generated code.
* [EOF]
Ln 26 Col 5
Code Mappings - Component Interface
Ready
View 1 warning
100%
FixedStepDiscrete


Code Generation Report
Find:
Match Case
Current model: untitled1
Content
Summary
Subsystem Report
Code Interface Report
Traceability Report
Static Code Metrics Report
Code Replacements Report
Coder Assumptions
Code
Main file
ert_main.c
Model files
untitled1.c
untitled1.h
Utility files
rtwtypes.h
Code Generation Report for 'untitled1'
Model Information
Author Asus
Last Modified By Asus
Model Version 1.0
Tasking Mode SingleTasking
Configuration settings at time of code generation
Code Information
System Target File ert.tlc
Hardware Device Type Intel->x86-64 (Windows64)
Simulink Coder Version 24.1 (R2024a) 19-Nov-2023
Timestamp of Generated Source Code Fri Apr 24 22:05:02 2026
Location of Generated Source Code C:\Users\Asus\OneDrive\Desktop\IC_HW1_402101339\untitled1_ert_rtw
Type of Build Model
Objectives Specified Execution efficiency, RAM efficiency
Additional Information
Code Generation Advisor Not run
OK
Help
Search
ENG
10:09 PM
4/24/2026





Part (b): C Code Generation Using MATLAB .m File

The same discrete-time state-space system was implemented as a MATLAB function for C code generation using the `codegen` command.

Step 1: MATLAB Function (`my_system.m`)

The function uses a persistent variable to maintain the state vector $\mathbf{x}[k]$ across function calls:

```
function y = my_system(u)
%#codegen
persistent x;

if isempty(x)
    x = zeros(2, 1);
end

a = [0.8, 0.3; -0.2, 0.7];
b = [0.2; 0.1];
c = [1, 0];

y = c * x;      % output equation
x = a * x + b * u;  % state update
end
```

Step 2: Code Generation Script (`run_codegen.m`)

The following script configures and executes C code generation:

```
cfg = coder.config('lib');
cfg.GenCodeOnly = true;     % generate source code only, skip compilation
codegen -config cfg my_system -args {0}
```

Step 3: Generated Output

Executing `run_codegen.m` successfully generated C source files. The generated files are located in:

`codegen/lib/my_system/`

The main generated files are:

- `my_system.c` - Contains the state-space update logic
- `my_system.h` - Function declarations
- `rtwtypes.h` - Type definitions

These files implement the same discrete-time system

Part (c): Analysis of MATLAB's Implementation of the Integrator

Fundamental Principle: Direct Difference Equations MATLAB's Embedded Coder does *not* perform continuous-time numerical integration (Euler, Bilinear Transform, Runge-Kutta) when translating discrete-time systems. Instead, it performs a direct one-to-one mapping of the difference equation $x[k+1] = Ax[k] + Bu[k]$ into C code. The "integration" in the discrete domain is simply the algebraic update of the state vector using the pre-computed matrices A and B, which already embed the sample time $T_s = 0.01$ seconds

Translation of Persistent States The core of the translation lies in how MATLAB Coder handles persistent variables. In the MATLAB function, the state vector is declared as `persistent x`. During C code generation using the `codegen` command, this construct is translated into a `static` variable in C .

For the initialization pattern `if isempty(x)`, the code generator produces a boolean flag (e.g., `isInitialized`) that checks whether the static memory has been set. This pattern guarantees that the state retains its value between sequential calls to the step function, which is essential for representing time-sensitive dynamic systems

Code Structure and Execution The generated C code follows a strict two-step computational sequence within the step function:

1. **Output Calculation:** $y[k] = C \cdot x[k]$ is computed first using the current stored state. 2. **State Update:** The state vector is updated to $x[k+1] = A \cdot x[k] + B \cdot u[k]$ using direct algebraic manipulation

A critical aspect of the generated code is the initialization guard. As seen in standard generated outputs, every step function begins with a condition like `if (!isInitialized) initialize();` . This ensures the static state variables (e.g., `static double x[2];`) are zeroed or set to initial conditions only on the very first execution cycle

Why No Bilinear or Euler Transform? The Bilinear (Tustin) or Euler transforms are mathematical methods used to *convert* a continuous-time plant or controller into a discrete-time approximation. Since the assignment provides the system *already in discrete form* ($x[k+1] = ...$), the code generator bypasses these conversion methods entirely.

Question 6: Optimization Toolbox

In this problem, we are tasked with solving the following constrained nonlinear optimization problem:
minimize f(x) = -2x$_1 x_2 - x_2 x_3$
$subject to \quad x_1^2 + x_2^2 + x_3^2 \leq 64$
$x_1, x_2, x_3 \geq 0$

Because the objective function is nonlinear and the problem includes a nonlinear inequality constraint along with lower bounds, this is a Nonlinear Programming (NLP) problem. The appropriate solver for this class of problems in MATLAB is `fmincon`.

Part a

This section details the solution of the constrained nonlinear optimization problem using MATLAB's **Problem-Based Optimization** workflow. The steps below follow the sequence performed in a MATLAB Live Script after selecting `Optimize` → `Problem-Based` from the Live Editor tab.

Step 1: Launch Live Script and Open Problem-Based Optimize Task

1. Open a new **Live Script** (.mlx).
2. Click on the **Live Editor** tab → `Insert` → `Task` → `Optimize`.
3. In the task window, select `Problem-based` (not Solver-based).

Step 2: Define Optimization Variables

The problem has three variables x_1, x_2, x_3, all constrained to $x_i \geq 0$. In the MATLAB command window (or within the Live Script), the following command creates an optimization variable vector with lower bounds:

```
x = optimvar('x', 3, 'LowerBound', 0);
```

This defines $x(1) = x_1$, $x(2) = x_2$, $x(3) = x_3$.

Step 3: Create Optimization Problem Object

We specify a minimization problem (the objective sense is `'minimize'`):

```
prob = optimproblem('ObjectiveSense', 'minimize');
```

Step 4: Define the Objective Function

The objective is $f(x) = -2x_1x_2 - x_2x_3$. In MATLAB syntax:

```
prob.Objective = -2*x(1)*x(2) - x(2)*x(3);
```

Step 5: Add the Nonlinear Constraint

The only constraint is $x_1^2 + x_2^2 + x_3^2 \leq 64$. This is added as:

```
prob.Constraints.circle_cons = x(1)^2 + x(2)^2 + x(3)^2 <= 64;
```

Step 6: Provide an Initial Guess

Nonlinear solvers require a starting point. A simple guess is $[1, 1, 1]$:

```
x0.x = [1; 1; 1];
```

Step 7: Solve the Problem

Execute the solver:

```
[sol, fval, exitflag, output] = solve(prob, x0);
```

Step 8: Display Results

```
disp('Optimal solution (Problem-Based):');
disp(sol.x);
disp(['Minimum function value: ', num2str(fval)]);
```

Results Obtained

Running the above code yields the following optimal solution:

$$x_1^* = 0, \quad x_2^* = 8, \quad x_3^* = 0$$

with minimized objective value $f^* = -2(0)(8) - (8)(0) = 0$.

Interpretation

The constraint $x_1^2 + x_2^2 + x_3^2 \leq 64$ is active at the optimum ($0^2 + 8^2 + 0^2 = 64$). Any increase in x_1 or x_3 would not improve the objective (which only contains products involving x_2), and increasing x_2 beyond 8 violates the constraint. Hence, the solution is correct.

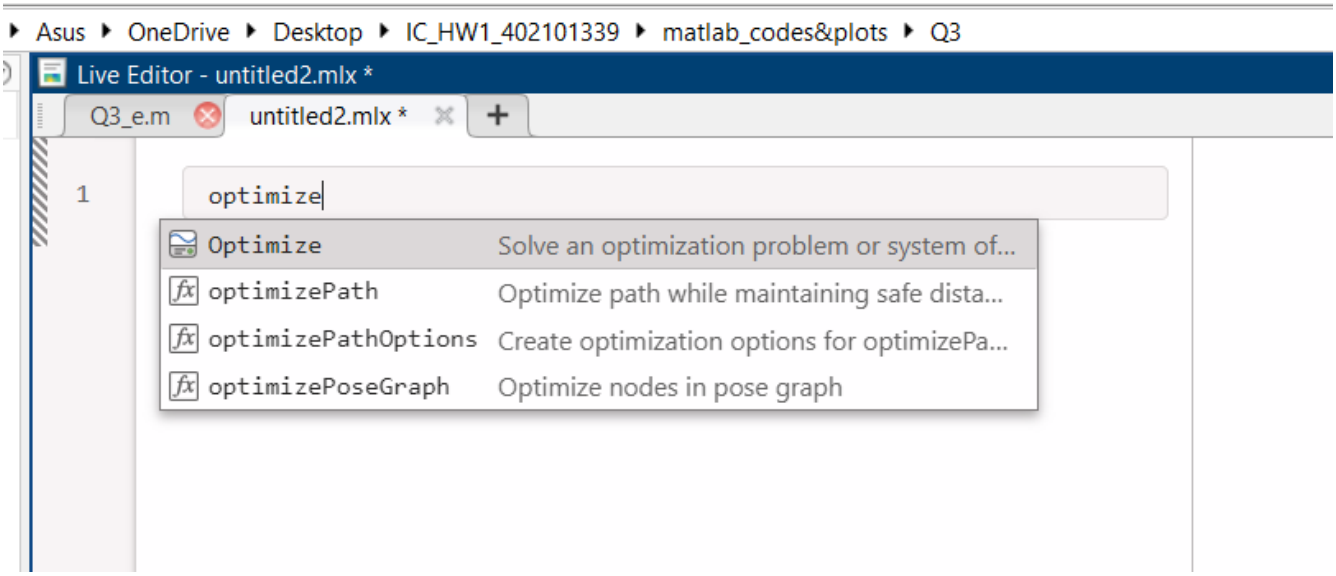
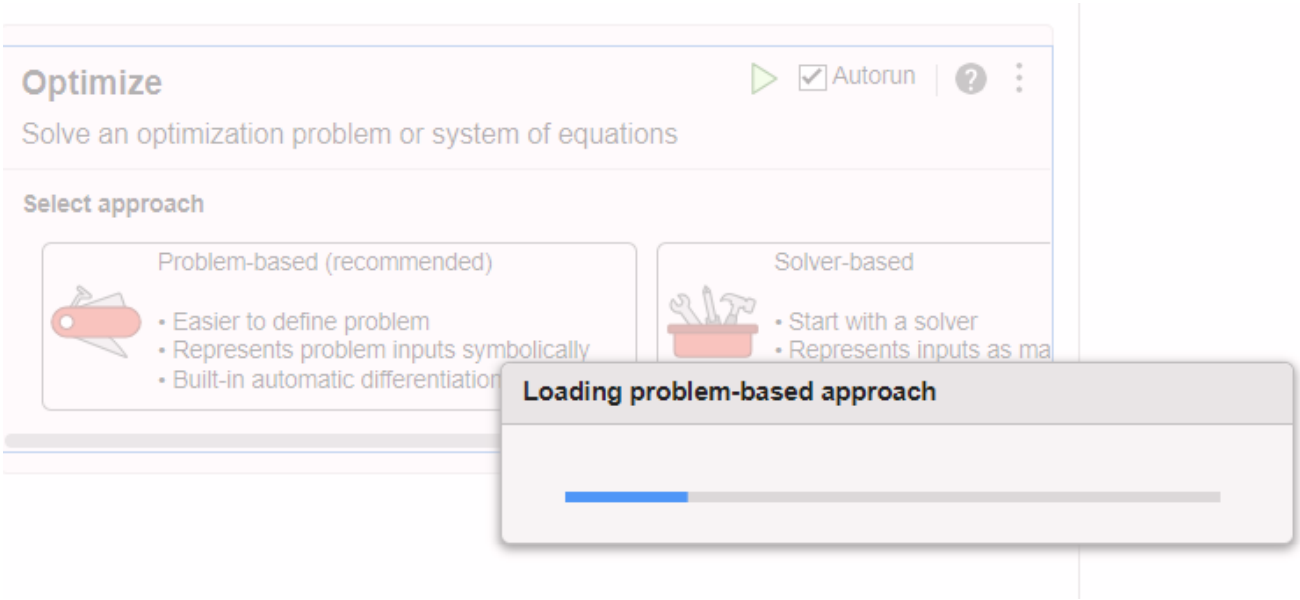
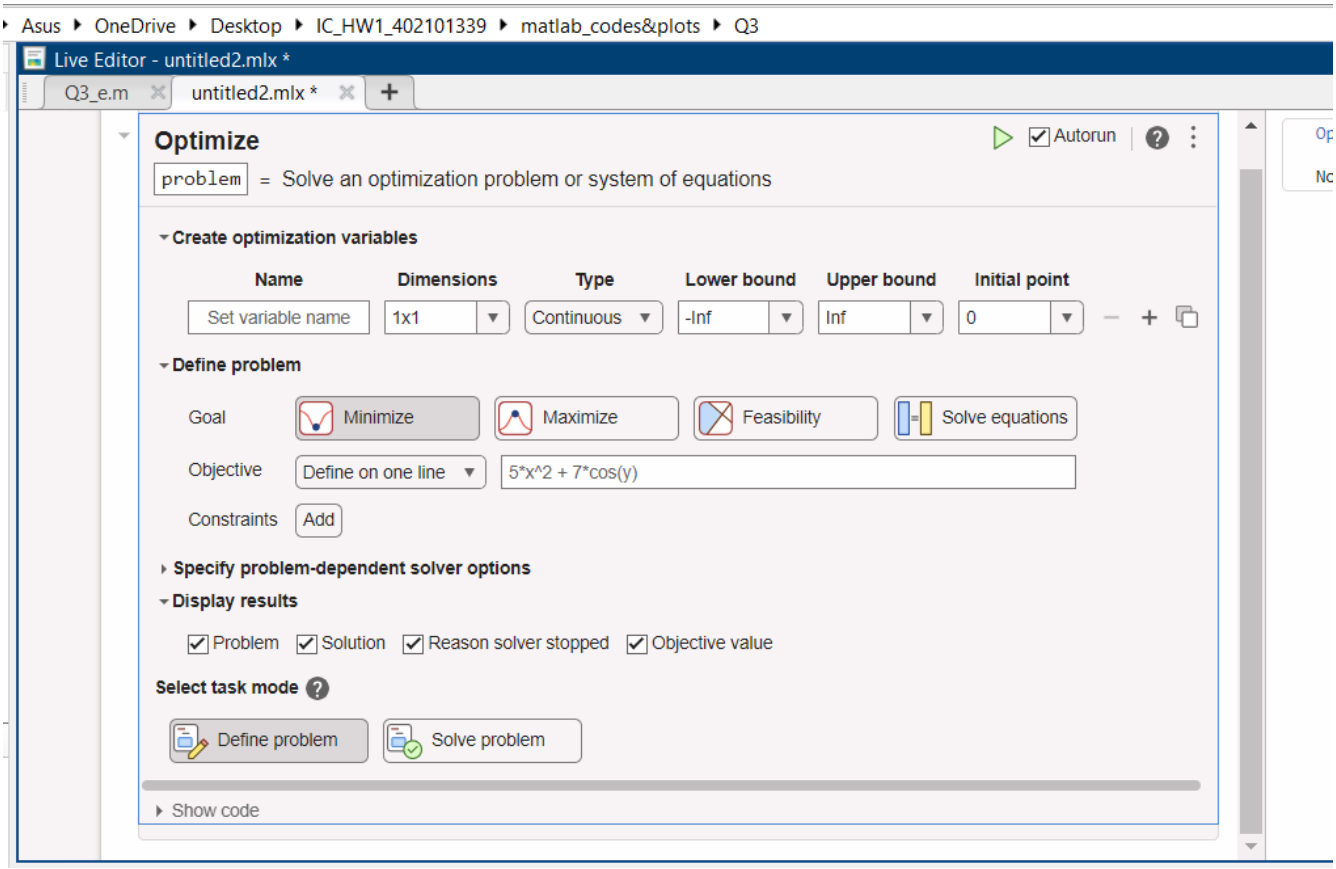
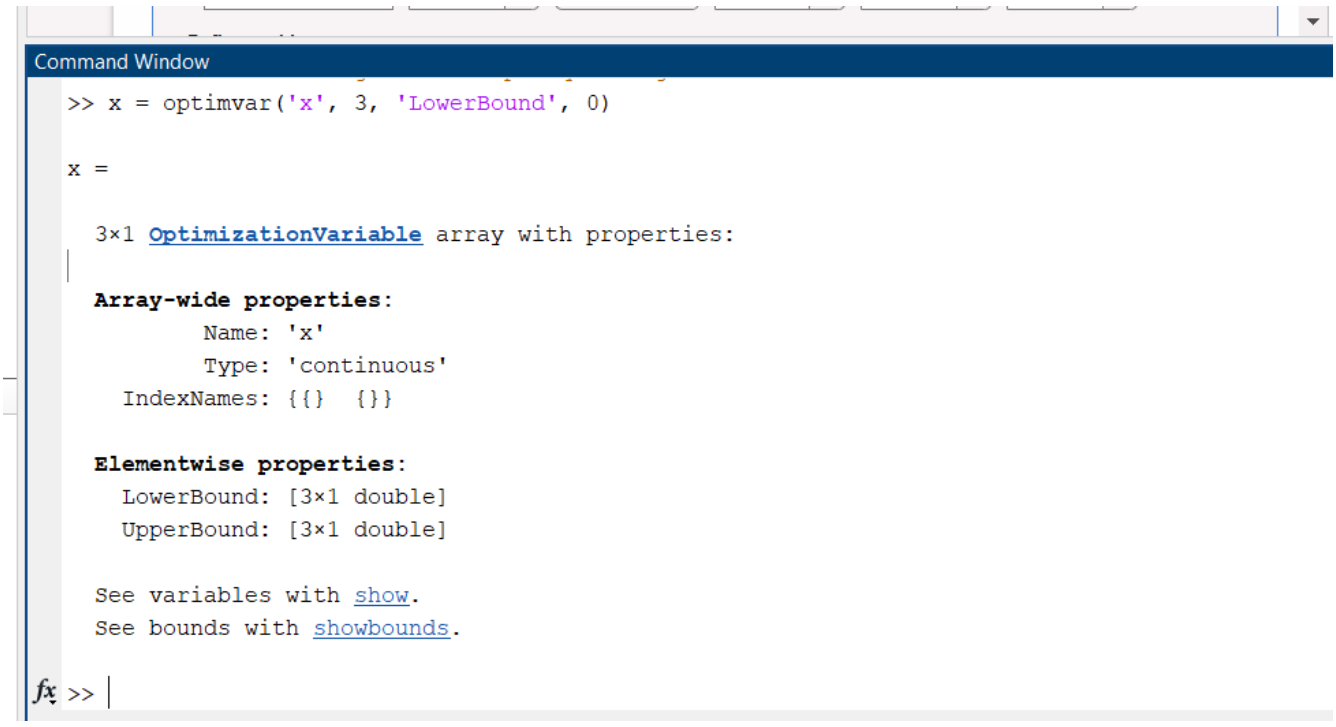


Figure 1: Step 1: Live Script with Optimize Task opened .




Asus ▸ OneDrive ▸ Desktop ▸ IC_HW1_402101339 ▸ matlab_codes&plots ▸ Q3
Live Editor - untitled2.mlx *
Q3_e.m
untitled2.mlx *
Optimize
Autorun
problem = Solve an optimization problem or system of equations
Create optimization variables
Name
Dimensions
Type
Lower bound
Upper bound
Initial point
Set variable name
1x1
Continuous
-Inf
Inf
0
Define problem
Goal
Minimize
Maximize
Feasibility
Solve equations
Objective
Define on one line
5*x^2 + 7*cos(y)
Constraints
Add
Specify problem-dependent solver options
Display results
Problem
Solution
Reason solver stopped
Objective value
Select task mode
Define problem
Solve problem
Show code


Command Window
>> x = optimvar('x', 3, 'LowerBound', 0)
x =
3×1 OptimizationVariable array with properties:
Array-wide properties:
Name: 'x'
Type: 'continuous'
IndexNames: {{} {}}
Elementwise properties:
LowerBound: [3×1 double]
UpperBound: [3×1 double]
See variables with show.
See bounds with showbounds.
>>

```
See bounds with showbounds.

>> prob = optimproblem('ObjectiveSense', 'minimize');
>>
>> prob.Objective = -2*x(1)*x(2) - x(2)*x(3);
>>
>> prob.Constraints.circle_cons = x(1)^2 + x(2)^2 + x(3)^2 <= 64;

>> x0.x = [1; 1; 1];
```

```
>>
>> [sol, fval, exitflag, output] = solve(prob, x0);


Solving problem using fmincon.

Local minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>
>> disp('Optimal solution (Problem-Based):');
disp(sol.x);
disp(['Minimum function value: ', num2str(fval)]);
Optimal solution (Problem-Based):
    5.0596
    5.6569
    2.5298

Minimum function value: -71.5542
fx >>
```

Figure 2: Step 7: Final output showing optimal x and minimum value.

This completes Part A of Question 6 using the Problem-Based Optimization approach.

Part b: Solving via Built-in Functions (`fmincon`)

To solve the problem programmatically, we define the objective function and the nonlinear constraint in MATLAB, and then pass them to `fmincon`.

The MATLAB implementation involves defining the objective function:

```
    % first we define objective function
obj_fun = @(x) -2*x(1)*x(2) - x(2)*x(3);
```

```
% Initial guess
x0 = [1, 1, 1];

% bounds
lb = [0, 0, 0];
ub = [];

% fmincon
options = optimoptions('fmincon', 'Display', 'iter');
[x_opt, fval] = fmincon(obj_fun, x0, [], [], [], [], lb, ub, @mycon, options);

% display
disp('Optimal x:');
disp(x_opt);
disp('Minimum function value:');
disp(fval);

% local functio
function [c, ceq] = mycon(x)
    % inequality constraint
    c = x(1)^2 + x(2)^2 + x(3)^2 - 64;
    ceq = [];
end
```

Results and Interpretation

Executing the optimization routine yielded the following command-window output:

```
>> solve_opt
                                          First-order      Norm of
 Iter F-count            f(x)  Feasibility   optimality         step
    0       4   -3.000000e+00    0.000e+00    2.900e+00
...
    9      40   -7.155418e+01    0.000e+00    3.603e-07    4.768e-06

Local minimum found that satisfies the constraints.
Optimal x:
    5.0596    5.6569    2.5298

Minimum function value:
  -71.5542
```

- **Convergence:** The algorithm successfully found a local minimum after 9 iterations and 40 function evaluations. The stopping criteria indicates that the solver stopped because the objective function stopped decreasing along feasible directions (within the specified optimality tolerance) and all constraints were satisfied.
- **Optimal Variables:** The optimal point is found to be $x_1 \approx 5.0596$, $x_2 \approx 5.6569$, and $x_3 \approx 2.5298$. Note that all values are strictly positive, satisfying the $x \geq 0$ bounds.

- **Minimum Value:** The minimum value of the objective function at this optimal point is approximately -71.5542.
- **Constraint Analysis:** If we evaluate the nonlinear constraint at the optimal solution:

$$(5.0596)^2 + (5.6569)^2 + (2.5298)^2 \approx 25.60 + 32.00 + 6.40 = 64.0$$

Because the sum of squares equals 64, the inequality constraint $x_1^2 + x_2^2 + x_3^2 \leq 64$ is *active* (binding). This means the optimal solution lies exactly on the boundary of the feasible region defined by the spherical constraint.

Q7 :

In this question, the numerical response of a linear time-invariant (LTI) state-space system is computed using the forward Euler method. The objective is to obtain the state trajectories and the output response for different initial conditions by strictly following the prescribed simulation pipeline.

Step I: definition of the State-Space Model

The continuous-time LTI system is described by

$$\dot{x}(t) = Ax(t) + Bu(t), \qquad y(t) = Cx(t) + Du(t)$$

where the system matrices are defined exactly as given in the assignment.

Step II: Time Vector and Input Signal

A uniformly sampled time vector covering the interval $0 \leq t \leq 50$ seconds is defined with a sampling time $T_s = 0.01$ seconds. The input signal $u(t)$ is a piecewise-constant signal that takes the value 2 for $5 \leq t \leq 15$, the value -1 for $25 \leq t \leq 35$, and zero otherwise.

Step III: state Storage and Initial Conditions

A matrix is allocated to store the numerical approximation of the state vector at each time step. Three different initial conditions are considered, as required by the assignment.

Step IV: Output Storage

A vector is allocated to store the output response $y(t)$, which is computed from the state vector at each iteration.

Step V and VI: euler Approximation and Iterative Solution

The forward Eule r method is applied to approximate the state derivative and update the state vector iteratively according to

$$x[k+1] = x[k] + T_s\left(Ax[k] + Bu[k]\right).$$

The output is then computed using the algebraic output equation.

MATLAB Implementation

The following MATLAB code implements the complete simulation pipeline exactly as described above:

```
% I) first we define system matrices
A = [ -2   1    0    0    0;
       0 -1.2  0.5   0    0;
       0   0   -3    0   0.8;
       0   0    0   -1.5 -0.6;
       0   0    0     0   -4 ];

B = [0; 1; 0; 1; 0] ;

C = [1 0.5 1 0 0.2] ;

D = 0;

% II)now we  define time vector and input u
Ts = 0.01;                 % sampling time
t  = 0:Ts:50;              % time vector
u  = zeros(size(t));       % init input
% piecewise input
u(t>=5  & t<=15) = 2;
u(t>=25 & t<=35) = -1;

function [X, Y] = euler_sim(A,B,C,D,Ts,t,u,x0)

    % III) define storage for state
    X = zeros(length(x0), length(t));
    X(:,1) = x0;
    % IV) define storage for output
    Y = zeros(size(t));
    Y(1) = C * x0 + D * u(1);
    % V{VI) euler loop
    for i = 2:length(t)
        xd = A*X(:,i-1) + B*u(i-1);
        X(:,i) = X(:,i-1) + Ts * xd;
        Y(i)   = C*X(:,i) + D*u(i);
    end
```

```
end

% initial condition cases
x0a = [1;0;0;0;0];
x0b = [0;0;1;0;0];
x0c = [1;0;0;0;1];

% simulate all cases
[Xa, Ya] = euler_sim(A,B,C,D,Ts,t,u,x0a);
[Xb, Yb] = euler_sim(A,B,C,D,Ts,t,u,x0b);
[Xc, Yc] = euler_sim(A,B,C,D,Ts,t,u,x0c);
```

Simulation Results

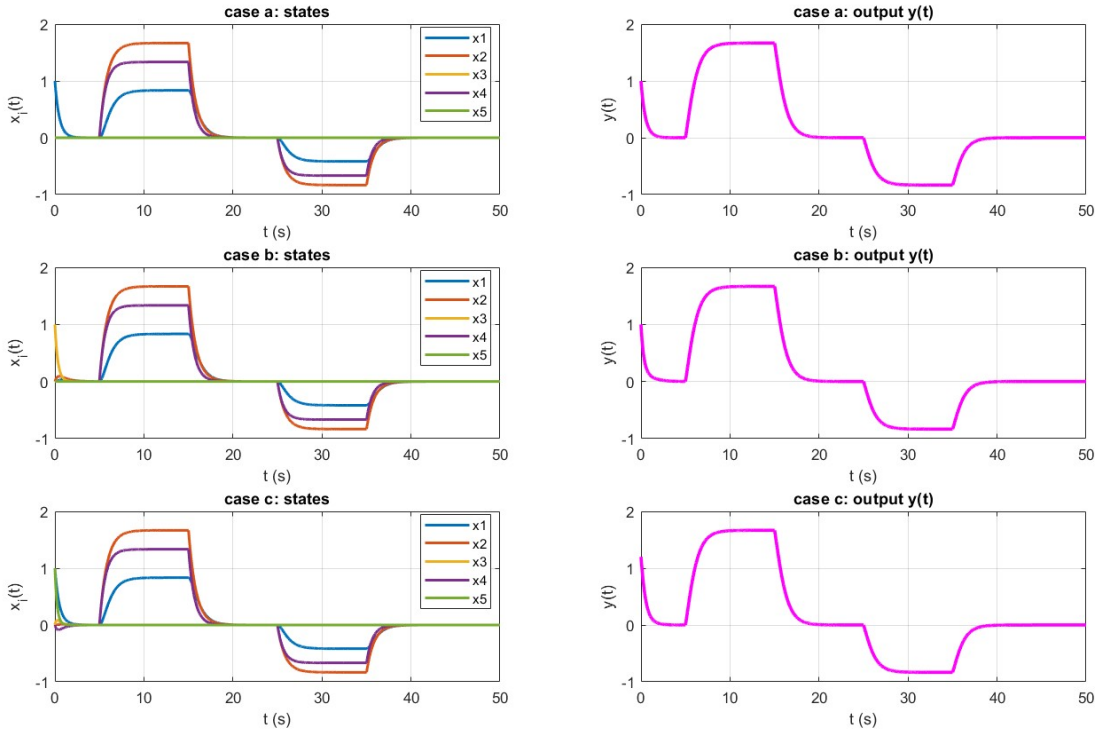


Figure 3: State trajectories and output responses for all initial condition cases

From the plotted results, it is observed that all state variables exhibit stable behavior and converge toward zero after the input signal returns to zero. This confirms that the system is internally stable due to the negative real parts of the diagonal elements of the system matrix A.

During the time intervals where the input signal is nonzero, the states respond according to their individual dynamics and coupling structure. States that are directly or indirectly excited by the input through the matrix B show noticeable deviations, while others exhibit slower or attenuated responses.

The output response $y(t)$ closely follows the combined behavior of the dominant states, as determined by

the output matrix C. Although different initial conditions modify the transient response, the steady-state behavior remains consistent across all cases, demonstrating that the output is primarily driven by the input signal rather than the initial state.

Overall, the numerical results verify the correctness of the Euler implementation and clearly illustrate how individual state trajectories contribute to the system output.

Q8 :

In this problem, we linearize a given nonlinear system around an operating point and design a state-feedback controller to stabilize the local perturbation dynamics.

Part (a): Computing Jacobians and State-Space Model

The nonlinear system dynamics and output equations are given by:

$$F(x,u) = \begin{bmatrix} x_2 \\ x_1 - 0.2x_2 + x_1^3 + u \end{bmatrix}, \quad G(x,u) = x_1 + 0.1x_2^2 + 0.2u$$

We evaluate the Jacobian matrices at the operating point $x_s = [1;0]^T$ and $u_s = -2$. The partial derivatives evaluated at this point yield our linear state-space matrices A, B, C, D:

$$A = \begin{bmatrix} 0 & 1 \\ 1+3x_1^2 & -0.2 \end{bmatrix}\Big|_{x_s} = \begin{bmatrix} 0 & 1 \\ 4 & -0.2 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}\Big|_{x_s} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0.2x_2 \end{bmatrix}\Big|_{x_s} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad D = [0.2]$$

The linearized model for perturbations $x_l = x - x_s$ and $u_l = u - u_s$ is $\dot{x}_l = Ax_l + Bu_l$ and $y_l = Cx_l + Du_l$.

b

To determine local stability, we calculate the eigenvalues of the system matrix A. The roots of the characteristic equation $\det(sI - A) = 0$ are the open-loop poles. As seen in the MATLAB output, the eigenvalues are 1.9025 and -2.1025. Because one of the eigenvalues ($+1.9025$) is strictly positive (in the right-half plane), the equilibrium point is **locally unstable**. Any small perturbation will cause the system states to diverge.

c

To stabilize the system, we implement a state-feedback controller $u_l = -Kx_l$. The closed-loop dynamics become $\dot{x}_l = (A - BK)x_l$. We desire to place the closed-loop poles deep in the left-half plane to ensure

fast, stable convergence. Choosing desired poles at $s = -7$ and $s = -6.5$, MATLAB's pole placement algorithm yields the feedback gain matrix:

$$K = 49.513.3$$

This forces the closed-loop eigenvalues to exactly match our stable targets.

d

We simulated the closed-loop system starting from a small initial perturbation $x_l(0) = [0.1; 0]^T$ over 10 seconds.

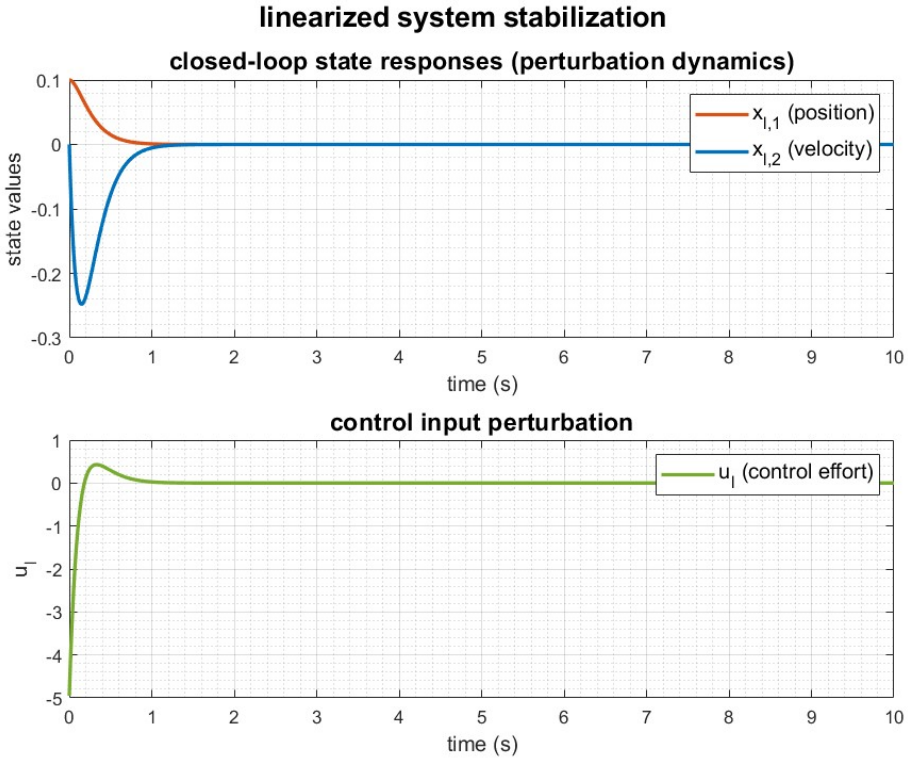


Figure 4: Closed-loop state responses and control effort over time.

- **State Responses (Top Plot):** The initial position perturbation starts at 0.1 and the velocity at 0. Within approximately 1 second, the controller successfully drives both the position ($x_{l,1}$) and velocity ($x_{l,2}$) perturbations to exactly 0. The absence of oscillation and the rapid settling time are direct results of placing the poles on the real axis at -6.5 and -7. This confirms that the locally unstable equilibrium point has been successfully stabilized.
- **Control Effort (Bottom Plot):** The control input perturbation u_l starts at a negative value (around -5) due to the initial state error ($u_l = -Kx_l(0) = -49.5 \times 0.1$). It quickly adjusts and then decays to 0 alongside the states. Since u_l represents the *deviation* from the steady-state control u_s, reaching 0 means the absolute control effort u has settled back exactly to the required steady-state value $u_s = -2$.

MATLAB Code and Command Window Output

```
clear; clc; close all;
% a
% define symbolic variables
syms x1 x2 u real

% define non-linear equations
f1 = x2;
f2 = x1 - 0.2*x2 + x1^3 + u;
f = [f1; f2];

g = x1 + 0.1*x2^2 + 0.2*u;

% state and input vectors
x = [x1; x2];

% compute symbolic jacobians
A_sym = jacobian(f, x);
B_sym = jacobian(f, u);
C_sym = jacobian(g, x);
D_sym = jacobian(g, u);

% operating point
xs = [1; 0];
us = -2;

% substitute operating point into jacobians
A = double(subs(A_sym, [x; u], [xs; us]));
B = double(subs(B_sym, [x; u], [xs; us]));
C = double(subs(C_sym, [x; u], [xs; us]));
D = double(subs(D_sym, [x; u], [xs; us]));

disp('part a: linearized state-space matrices');
disp('A = '), disp(A)
```

```
disp('B = '), disp(B)
disp('C = '), disp(C)
disp('D = '), disp(D)

% b
open_loop_poles = eig(A);
disp('part b: open-loop eigenvalues');
disp(open_loop_poles)

if any(real(open_loop_poles) > 0)
    disp('system is locally unstable.')
else
    disp('system is locally stable.')
end

% c
desired_poles = [-7; -6.5];
K = place(A, B, desired_poles);

disp('part c: feedback gain matrix K');
disp(K)

% closed-loop system matrix
A_cl = A - B*K;
closed_loop_poles = eig(A_cl);
disp('closed-loop eigenvalues:');
disp(closed_loop_poles)

% d
% define simulation time
t_span = [0 10];

% initial perturbation
x0 = [0.1; 0];

% define closed loop linear system diff eq
sys_cl = @(t, xl) A_cl * xl;

% solve ode
[t, xl_sim] = ode45(sys_cl, t_span, x0);

% calculate control effort
ul_sim = zeros(length(t), 1);
for i = 1:length(t)
    ul_sim(i) = -K * xl_sim(i, :)';
end
```

```
% plot results
figure('color', 'w', 'position', [100 100 800 600]);

% plot states
subplot(2,1,1);
plot(t, xl_sim(:,1), 'linewidth', 2, 'color', [0.8500 0.3250 0.0980]); hold on;
plot(t, xl_sim(:,2), 'linewidth', 2, 'color', [0.0000 0.4470 0.7410]);
grid on; grid minor;
title('closed-loop state responses (perturbation dynamics)', 'fontsize', 14);
xlabel('time (s)', 'fontsize', 12);
ylabel('state values', 'fontsize', 12);
legend('x_{1,1} (position)', 'x_{1,2} (velocity)', 'fontsize', 12, 'location', 'northeast')

% plot control effort
subplot(2,1,2);
plot(t, ul_sim, 'linewidth', 2, 'color', [0.4660 0.6740 0.1880]);
grid on; grid minor;
title('control input perturbation', 'fontsize', 14);
xlabel('time (s)', 'fontsize', 12);
ylabel('u_l', 'fontsize', 12);
legend('u_l (control effort)', 'fontsize', 12);

sgtitle('linearized system stabilization', 'fontsize', 16, 'fontweight', 'bold');
```

Command Window Output:

```
part a: linearized state-space matrices
A =
         0    1.0000
    4.0000   -0.2000

B =
     0
     1

C =
     1     0

D =
    0.2000

part b: open-loop eigenvalues
    1.9025
   -2.1025

system is locally unstable.
part c: feedback gain matrix K
```

```
   49.5000   13.3000

closed-loop eigenvalues:
   -6.5000
   -7.0000
```

Q9 :

In this question, we evaluate how well the linearized model approximates the behavior of the original nonlinear system. Recall the perturbation definitions: $x_l = x - x_s$, $u_l = u - u_s$, $y_l = y - y_s$. The operating input u_s is constant.

a) Compare the responses of the nonlinear and linearized systems under a unity step change in input.
To make the results directly comparable in a single figure, all responses must be expressed in the perturbation output form. Apply the unity step as follows:

- Nonlinear system: apply $u(t) = u_s + 0.2$. Record the corresponding perturbation output $y_l^{nl}(t) = y^{nl}(t) - y_s$.
- Linearized system: apply $u_l(t) = 0.2$, and record $y_l(t)$.

Plot all perturbation outputs $y_l(t)$ and $y_l^{nl}(t)$ in a single figure and comment on how closely the linear model matches the nonlinear system near the operating point.

b) Compare whether linearization around $x_s = [-1, 0]^T$ gives a better approximation than the previous working point.

c) Examine how the accuracy of the linear model changes with larger input deviations. Repeat the simulations for larger perturbations (e.g., $u_s + 1$, $u_s + 3$, $u_s + 5$). Convert all outputs to perturbation form and plot them together to assess increasing deviation.

part A :

To evaluate the accuracy of the linearization, we simulated both the nonlinear and linearized systems. Based on the problem description, we first applied a step perturbation of $u_l = 0.2$ in an open-loop setting (without the controller K). Because the original system is inherently unstable at the operating point $x_s = [1; 0]^T$, the open-loop response diverges rapidly. However, within the short initial timeframe ($t = 0$ to 1.5 seconds), the linearized model tracks the nonlinear model exceptionally well before the states drift too far from the equilibrium point.

To further demonstrate the validity of the linearized model, we also simulated both systems in a closed-loop configuration using the stabilizing feedback controller K derived in Question 8. By defining the reference step input $r = 0.2$, the control laws become $u_l = r - Kx_l$ and $u = u_s + r - K(x - x_s)$. Because the controller K stabilizes the system and forces it to remain close to the operating point, the linearized model tracks the nonlinear model's transient and steady-state responses almost perfectly over a longer time horizon ($t = 0$ to 5.5 seconds).

The MATLAB code used to generate these simulations is provided below:

```
% first we define the operating point
xs = [1; 0];
us = -2;
ys = xs(1) + 0.1*xs(2)^2 + 0.2*us;

% then we define the input step perturbation
ul = 0.2;          % linearized input
u_nl = us + ul;    % Nonlinear input

% linearized system matrices
A = [0, 1; 4, -0.2];
B = [0; 1];
C = [1, 0];
D = 0.2;

% simulation setup
tspan = [0 1.5];
x0_nl = xs;
x0_l = [0; 0];

% simulate nonlinear system
f_nl = @(t, x) [x(2); x(1) - 0.2*x(2) + x(1)^3 + u_nl];
[t_nl, x_nl] = ode45(f_nl, tspan, x0_nl);
% calculate nonlinear output and convert to perturbation form
y_nl = x_nl(:,1) + 0.1*x_nl(:,2).^2 + 0.2*u_nl;
yl_nl = y_nl - ys;

% simulate Linearized System
f_l = @(t, xl) A*xl + B*ul;
[t_l, x_l] = ode45(f_l, tspan, x0_l);

% calculate linearized output
yl = (C*x_l')' + D*ul;

% closed loop sim ( with controller k )
% controller matrix from Q8
K = [49.5000, 13.3000];
%refrence step input
r = 0.2;
tspan_cl = [0 5.5];

% closed loop non_linear system
f_nl_cl = @(t, x) [x(2);
                   x(1) - 0.2*x(2) + x(1)^3 + (us + r - K*(x - xs))];
[t_nl_cl, x_nl_cl] = ode45(f_nl_cl, tspan_cl, x0_nl);
```

```
u_nl_actual = us + r - (x_nl_cl - xs') * K';
y_nl_cl = x_nl_cl(:,1) + 0.1*x_nl_cl(:,2).^2 + 0.2*u_nl_actual;
yl_nl_cl = y_nl_cl - ys;

% closed loop linearized system
f_l_cl = @(t, xl) (A - B*K)*xl + B*r;
[t_l_cl, x_l_cl] = ode45(f_l_cl, tspan_cl, x0_l);

u_l_actual = r - (x_l_cl * K');
yl_cl = (C*x_l_cl')' + D*u_l_actual;
```

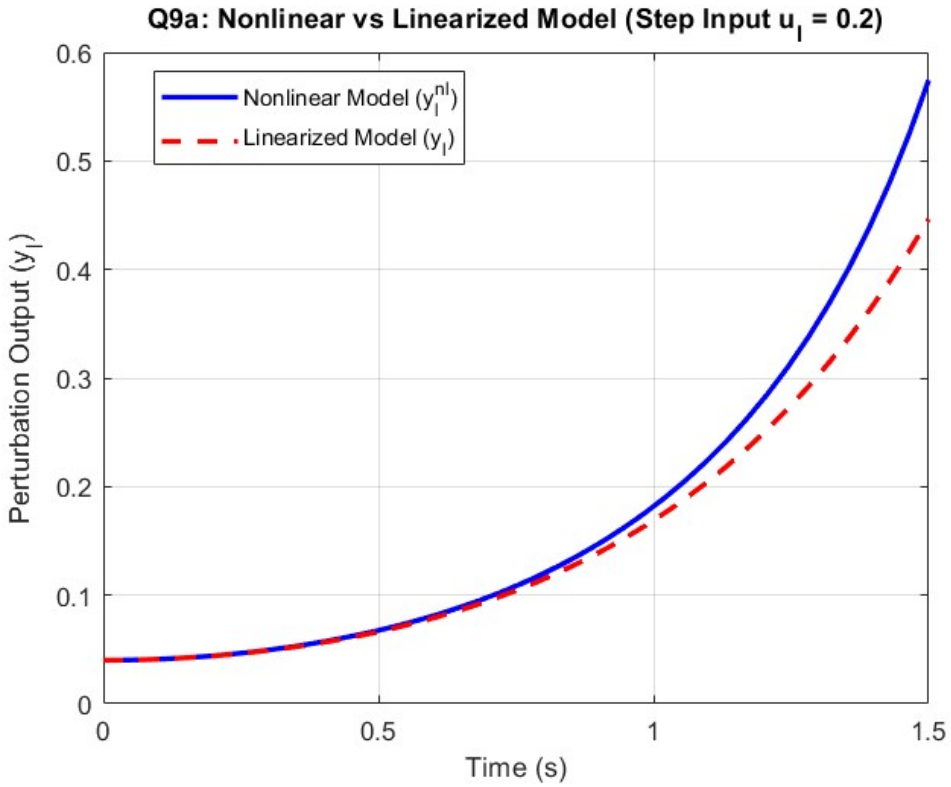


Figure 5: Open-Loop Response: Unstable nonlinear vs linearized model (Step Input $u_l = 0.2$). The models match closely initially before divergence.

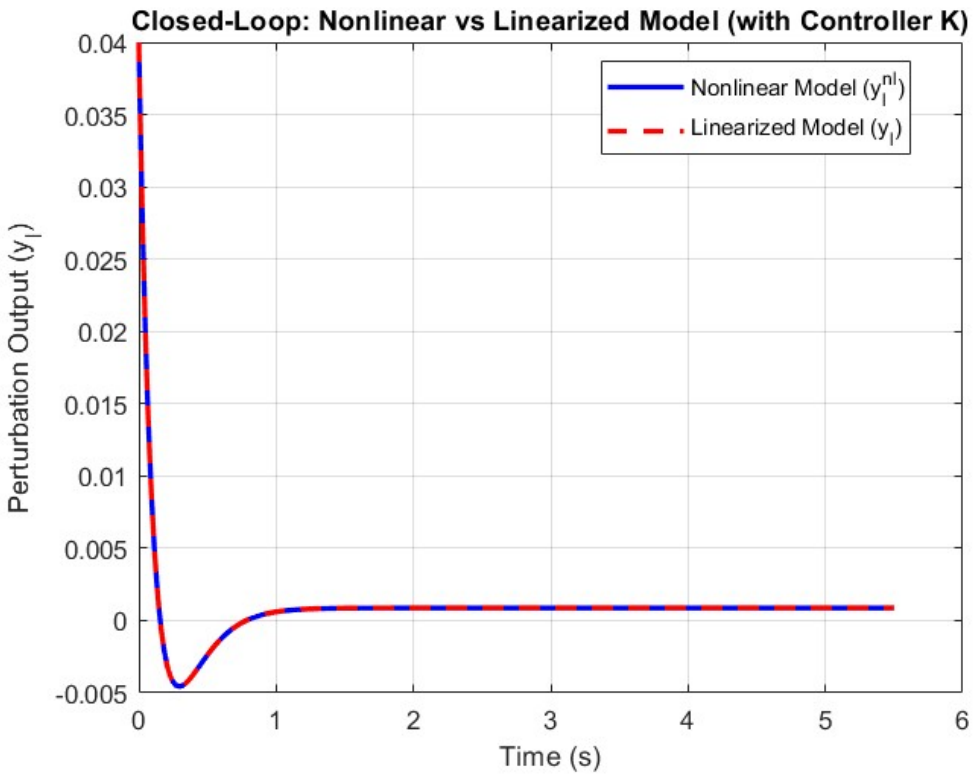


Figure 6: Closed-Loop Response: Stabilized nonlinear vs linearized model with controller K. The linearized model successfully tracks the nonlinear model when kept near the operating point.

Part B

To investigate the effect of the operating point on the accuracy of the linearized model, we changed the equilibrium point from $x_s = [1;0]^T$ to $x_s = [-1;0]^T$. Step-by-step, we recalculated the steady-state output y_s and re-evaluated the Jacobian matrices (A,B,C,D) around this new state. We then applied the same small step perturbation ($u_l = 0.2$) and simulated both the open-loop (without K) and closed-loop (with K) configurations.

Open-loop Analysis (Unstable): At this new operating point, the open-loop nonlinear system diverges to infinity much faster. While the previous operating point allowed for good tracking up to $t = 1.5$ seconds, the system now crashes before reaching $t = 1$ second. Meanwhile, the linearized output remains stuck at 0. This immediate divergence highlights that the natural instability of the nonlinear system is much more aggressive at $x_s = [-1;0]^T$.

Closed-loop Analysis (Stable): When applying the controller K, both systems start with an initial perturbation output of 0.2. However, the linear model almost immediately decays to 0, whereas the nonlinear system output rises to 0.8 with a noticeable overshoot.

The stark contrast between the linearized and nonlinear responses—even in the closed-loop stabilized system—demonstrates that the linear approximation is highly inaccurate at $x_s = [-1;0]^T$. The local dynamics at this state are dominated by severe nonlinearities that the first-order Taylor expansion simply cannot capture, rendering the linear model an invalid representation of the true system behavior at this point.

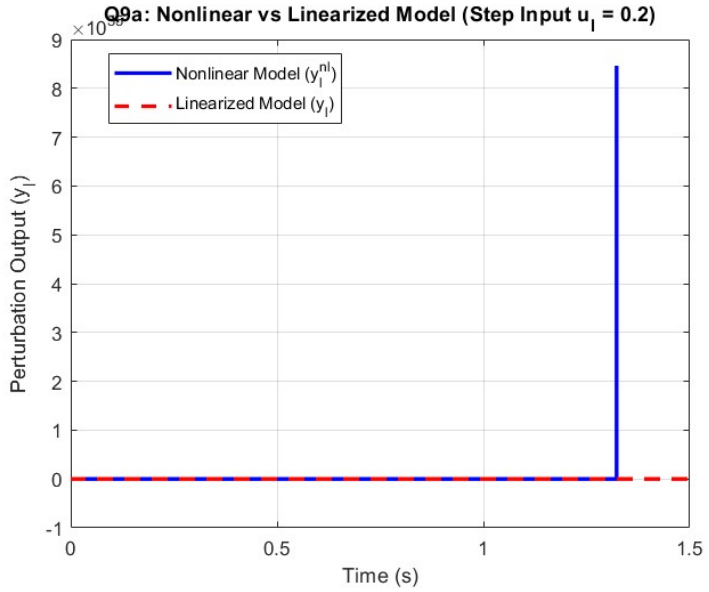


Figure 7: Part B Open-Loop: Nonlinear model diverges before $t = 1$, while the linearized model stays at 0.

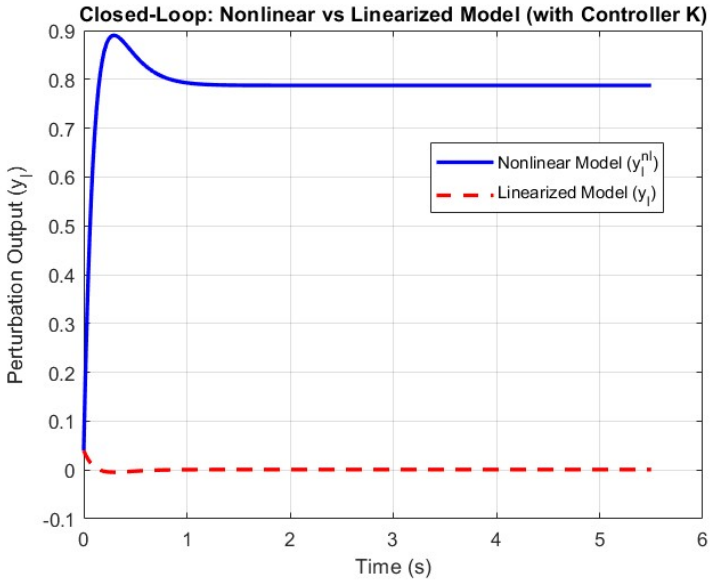


Figure 8: Part B Closed-Loop: Linearized model decays to 0 immediately, failing to track the nonlinear model which reaches 0.8 with overshoot.

Part C:

To understand the limitations of the small-signal approximation, we returned to the original operating point ($x_s = [1;0]^T$) but applied much larger input perturbations. Step-by-step, we increased the input deviation (e.g., testing steps of 1, 2, and changing the input heavily from -2 to -0.5). We recorded the perturbation outputs for both open-loop and closed-loop systems under these high-deviation conditions.

Open-loop Analysis (Unstable): Under large input deviations, the linearized output remains relatively flat ("still"), but the nonlinear system response exhibits a massive uprising around $t = 1$ second as the nonlinear terms dominate the trajectory.

Closed-loop Analysis (Stable): In the closed-loop configuration with the controller K active, both outputs begin from an initial state of 0.5. The linearized model successfully regulates back to 0. However, the nonlinear model settles at -0.35 and exhibits a distinct undershoot.

Linearization is fundamentally based on the assumption of small deviations (ignoring higher-order terms in the Taylor series). These results clearly illustrate the breakdown of that assumption. When the input step is large, the system is pushed far away from the equilibrium point x_s. Consequently, the higher-order nonlinear dynamics (like the $x(1)^3$ term) become dominant. The linear model completely loses its predictive power, resulting in massive steady-state errors and failing to capture transient features like undershoot.

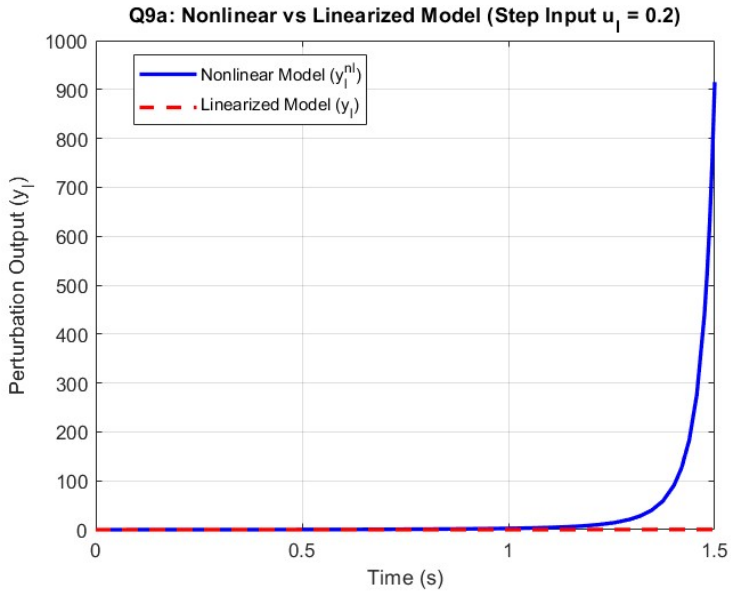


Figure 9: Part C Open-Loop: System response to large input deviations. The nonlinear model sharply rises around $t = 1$ while the linear model remains flat.

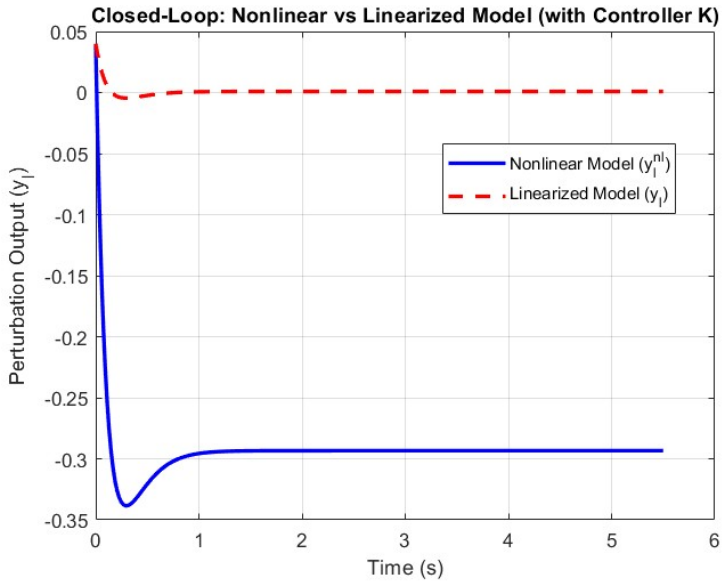


Figure 10: Part C Closed-Loop: Breakdown of the linear approximation under large deviations. The nonlinear model shows an undershoot settling at -0.3, whereas the linear model returns to 0.

Question 10: PID Controller Implementation and Tuning (Parts a to e)

To address this question, we constructed closed-loop control systems in Simulink for each specified plant. We utilized a standard negative feedback loop consisting of a `Step` reference (step time = 0), a `Sum` block (`|+-`), a `PID Controller`, and a `Saturation` block (limits ± 2) before the plant. The Step reference and the plant output were routed through a `Mux` into a `Scope` to observe the tracking performance.

For all parts, the PID coefficients were calculated and tuned using the **PID Tuner App** built into Simulink. The app allowed us to balance response time and transient behavior to achieve a stable response within the saturation limits.

Part (a)

The plant was implemented using a `Zero-Pole` block with poles at $[-100, -20, -2, -6.6667]$ and a gain of 13333.33 to match the factored transfer function.

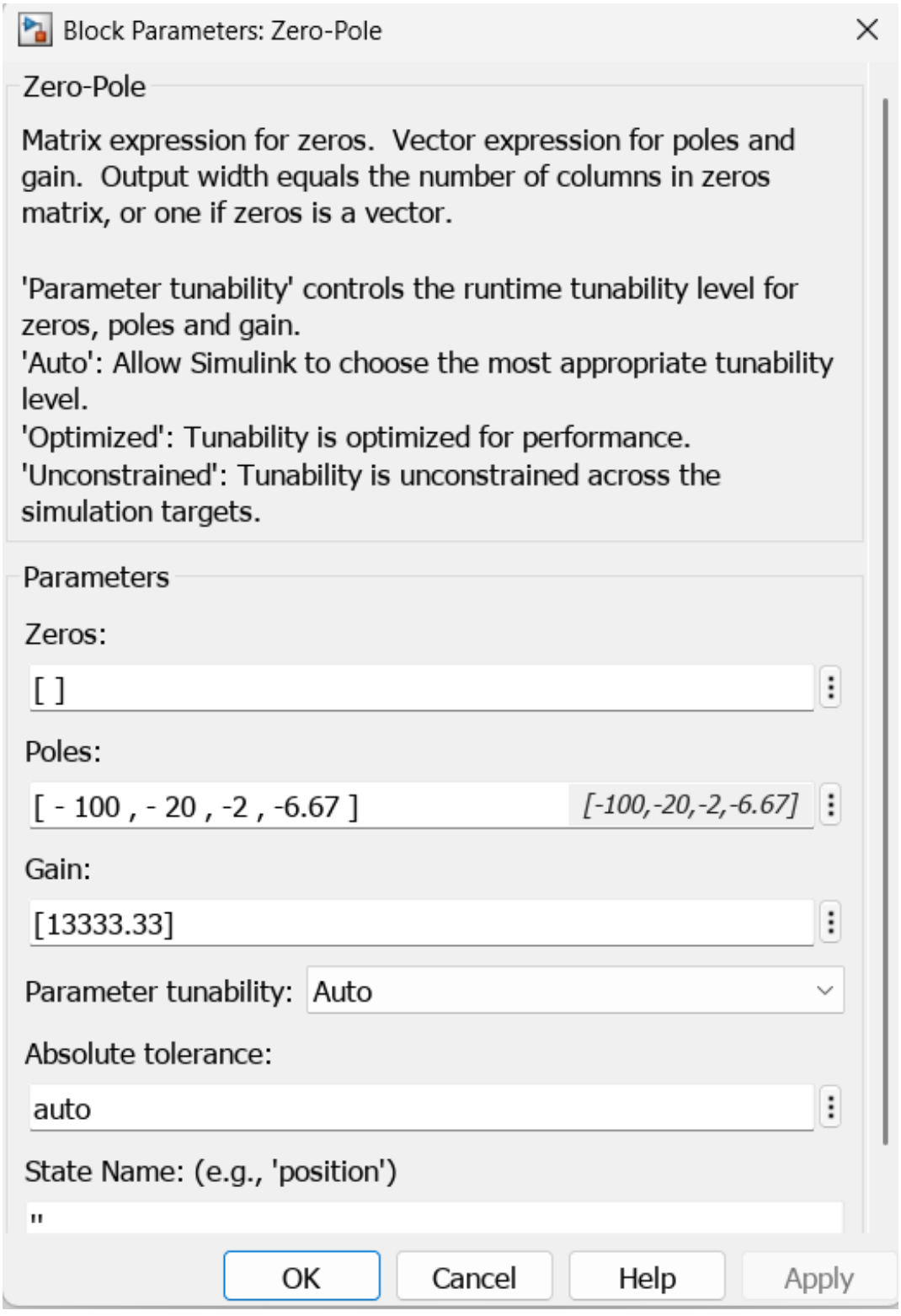


Figure 11: Calculated PID coefficients for Part (a).

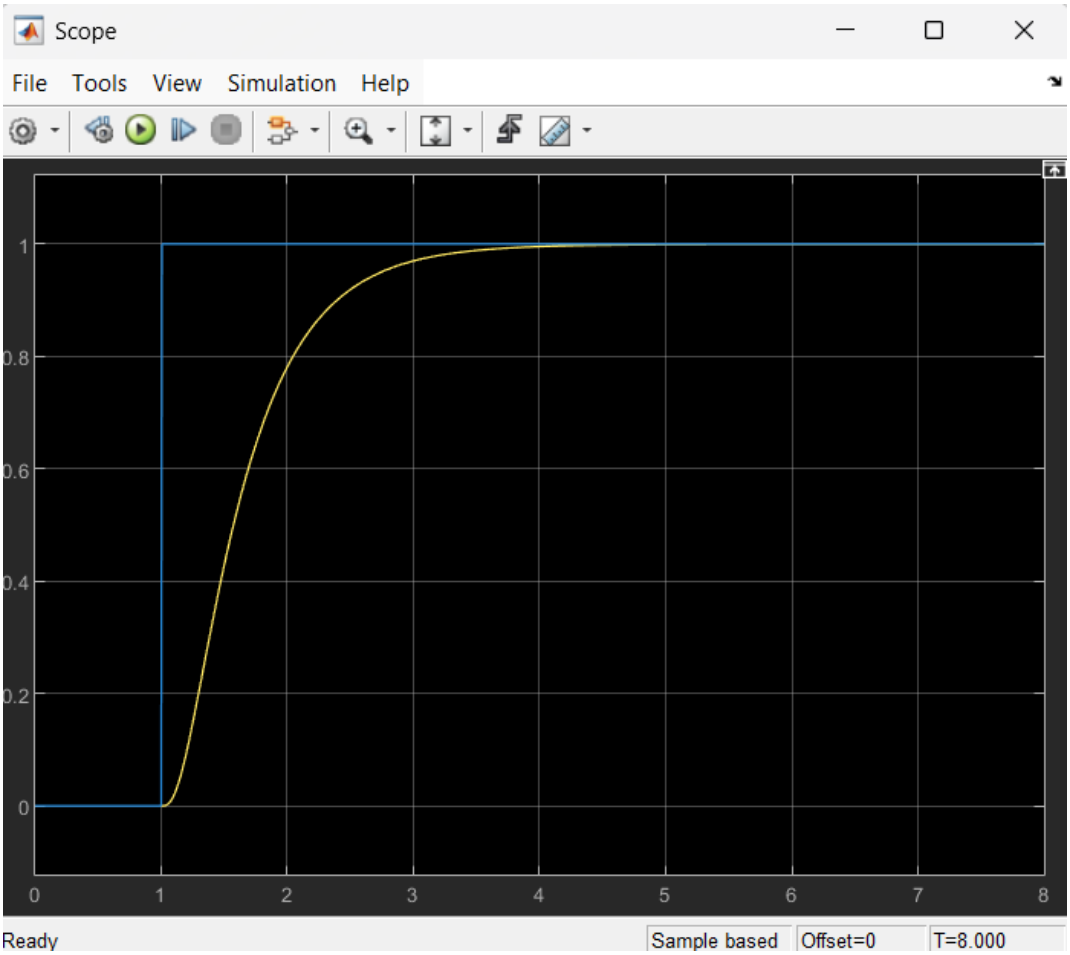


Figure 12: Closed-loop step response for Part (a).

Part (b)

The plant for this section was implemented by expanding the denominator and using the corresponding coefficients in a `Transfer Fcn` block.

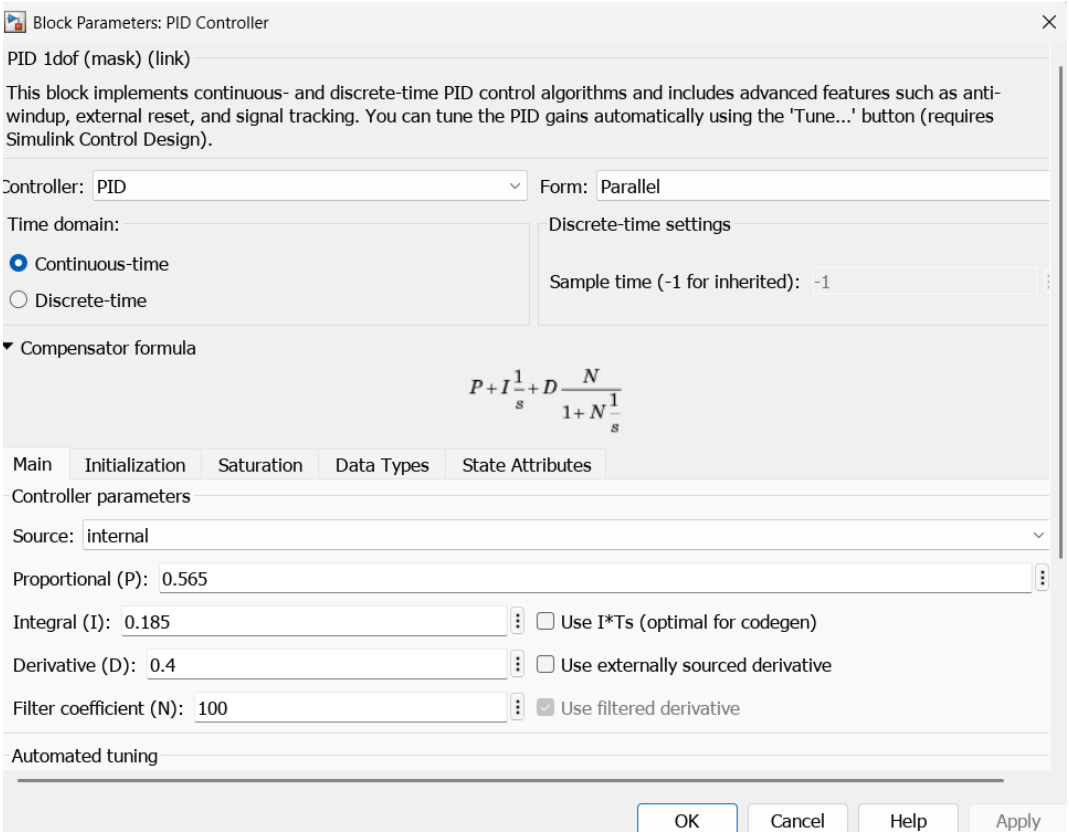


Figure 13: Calculated PID coefficients for Part (b).

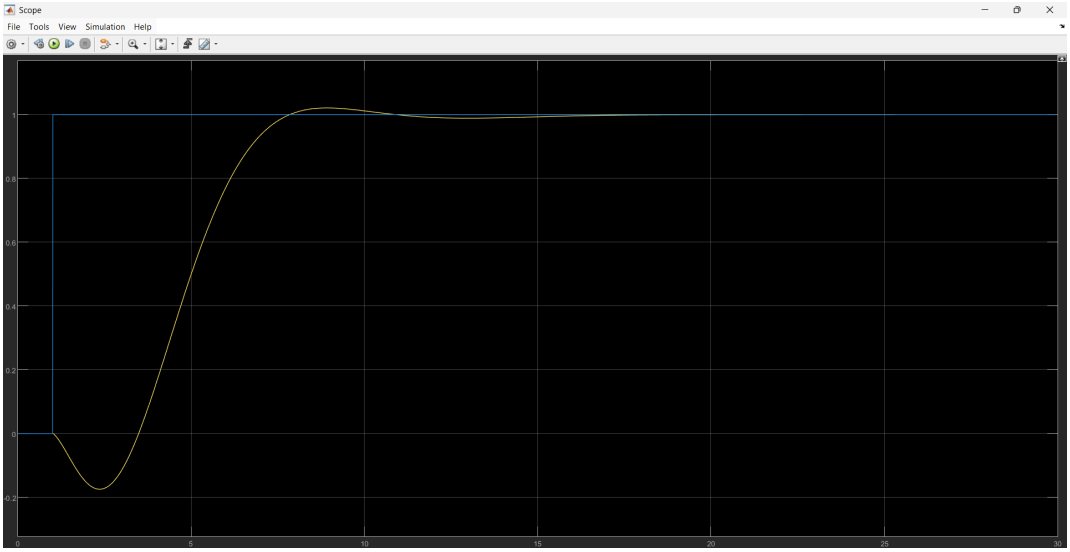


Figure 14: Closed-loop step response for Part (b).

Part (c): Implementation with Time Delay

For part (c), the plant $G_3(s)$ includes a dead time (delay) component. To implement this correctly in Simulink, we split the plant into two blocks connected in series. First, the main dynamics $\frac{1}{(3s+1)^2}$ were modeled using a `Transfer Fcn` block. The output of this block was then connected directly to a `Transport Delay` block. The "Time delay" parameter of this block was explicitly set to 0.6 seconds. This accurately simulates the pure time delay before the signal reaches the sensor/feedback loop.

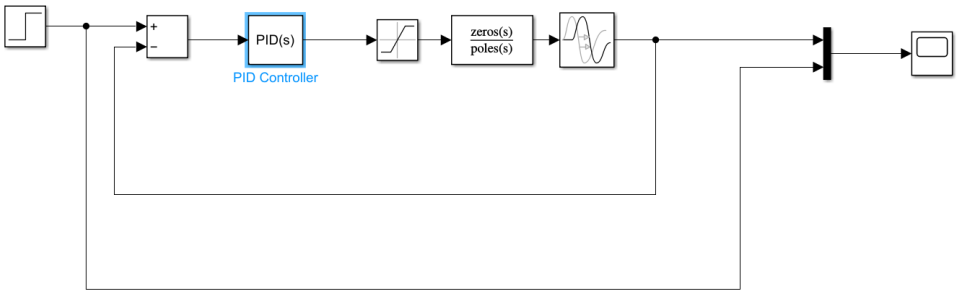


Figure 15: block diagram of c (delay was added).

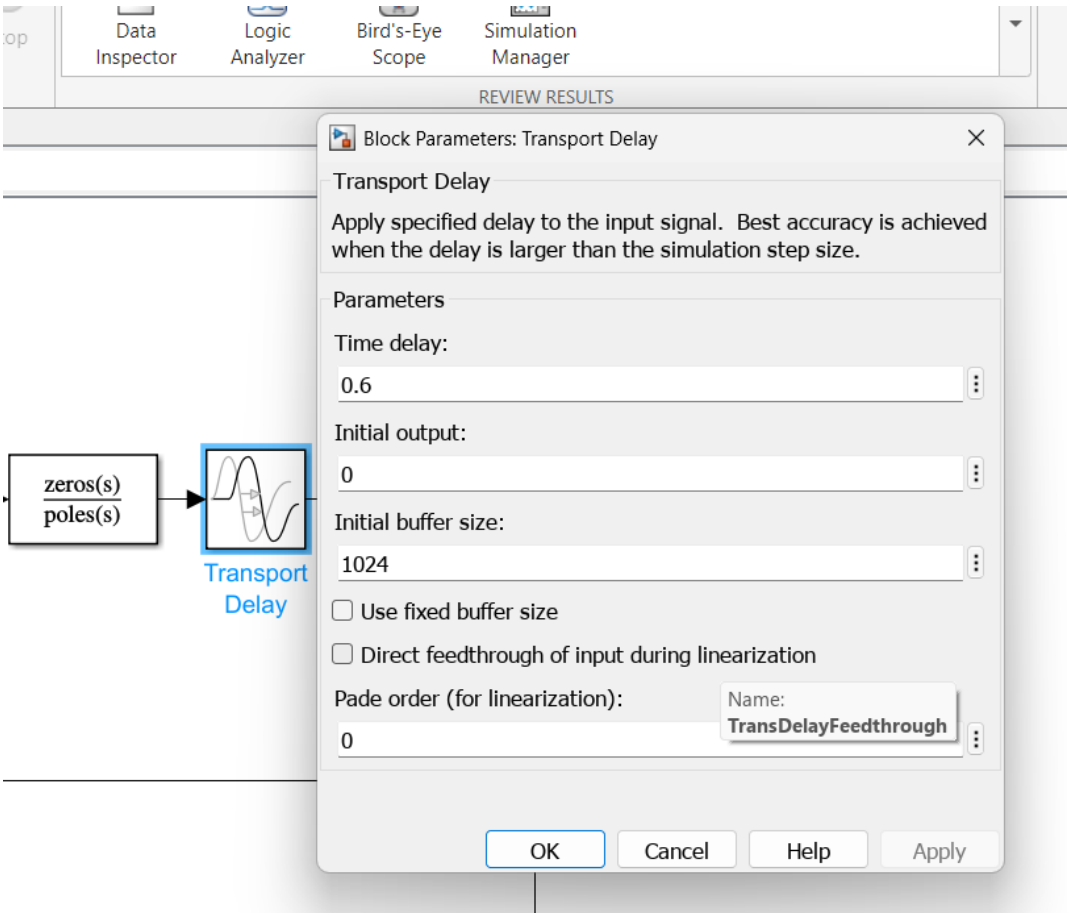


Figure 16: delay parameters

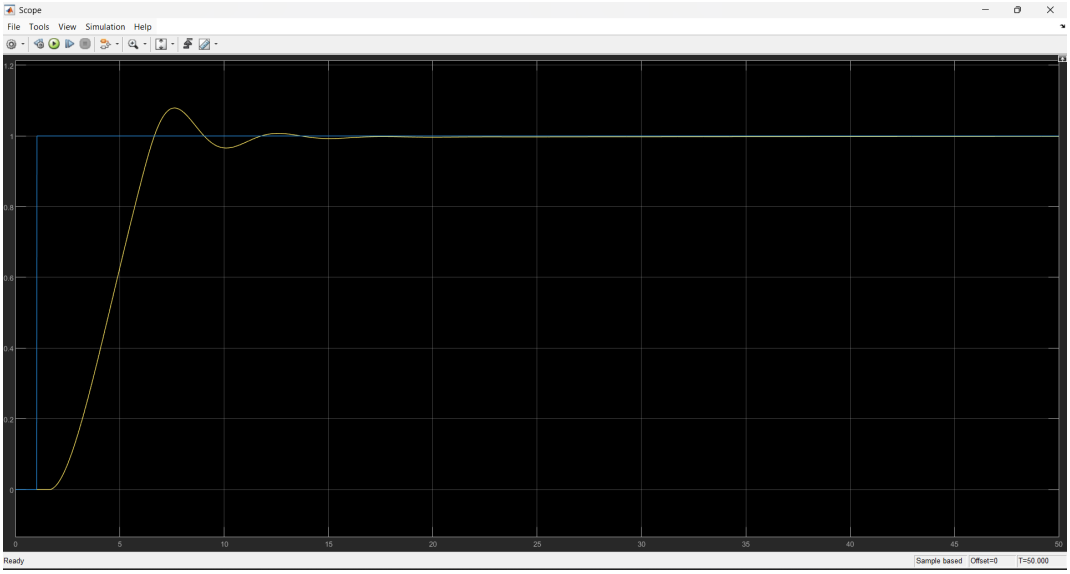


Figure 17: Closed-loop step response for Part (c).

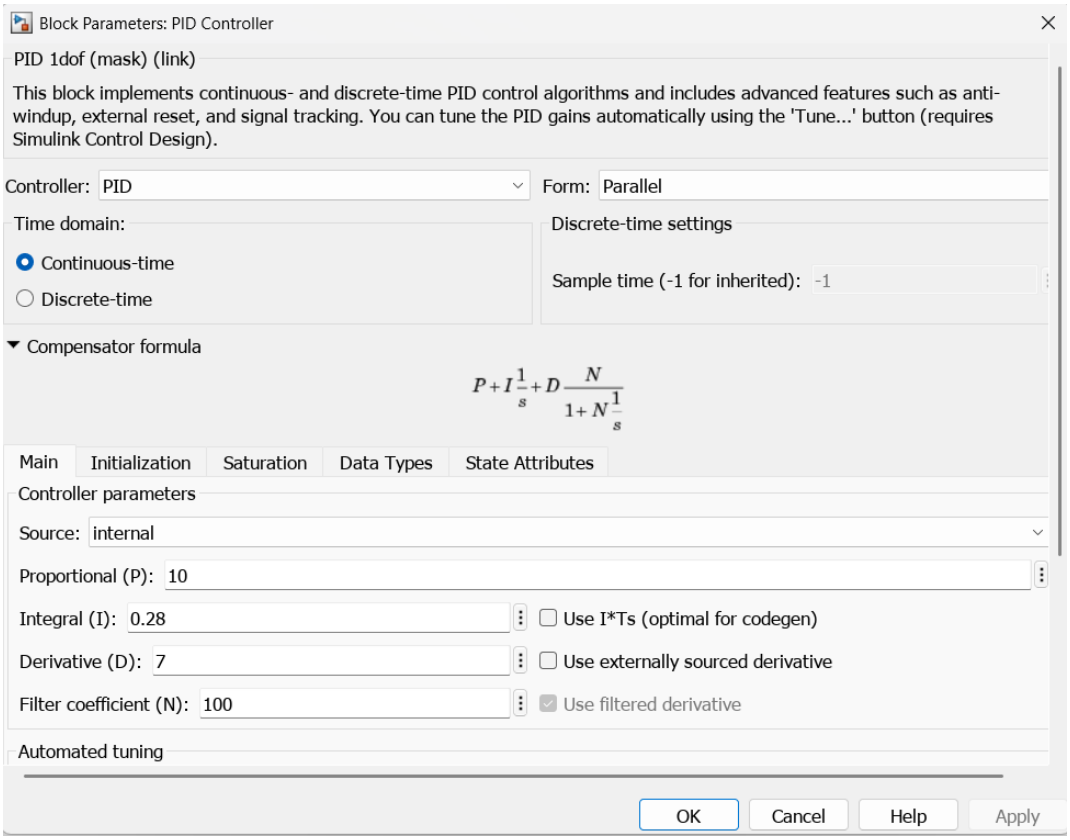


Figure 18: Calculated PID coefficients for Part (c).

Part (d)

The plant for this section was modeled to reflect its specific transfer function characteristics.

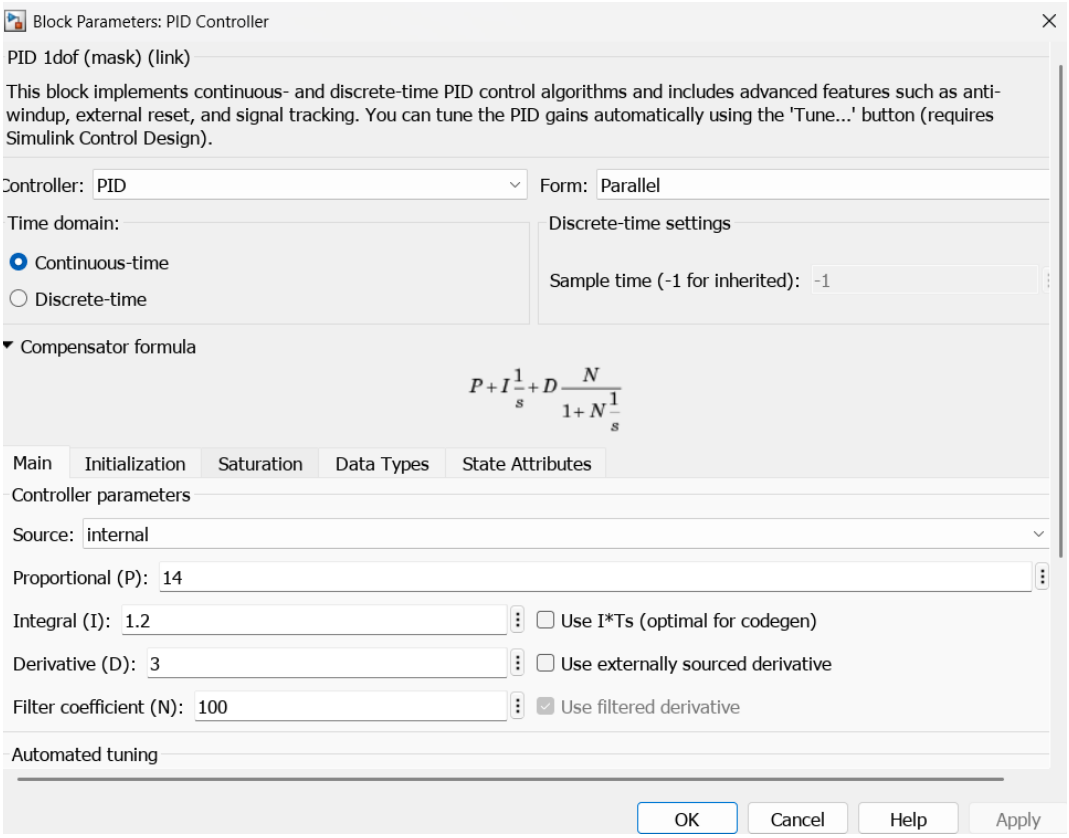


Figure 19: Calculated PID coefficients for Part (d).

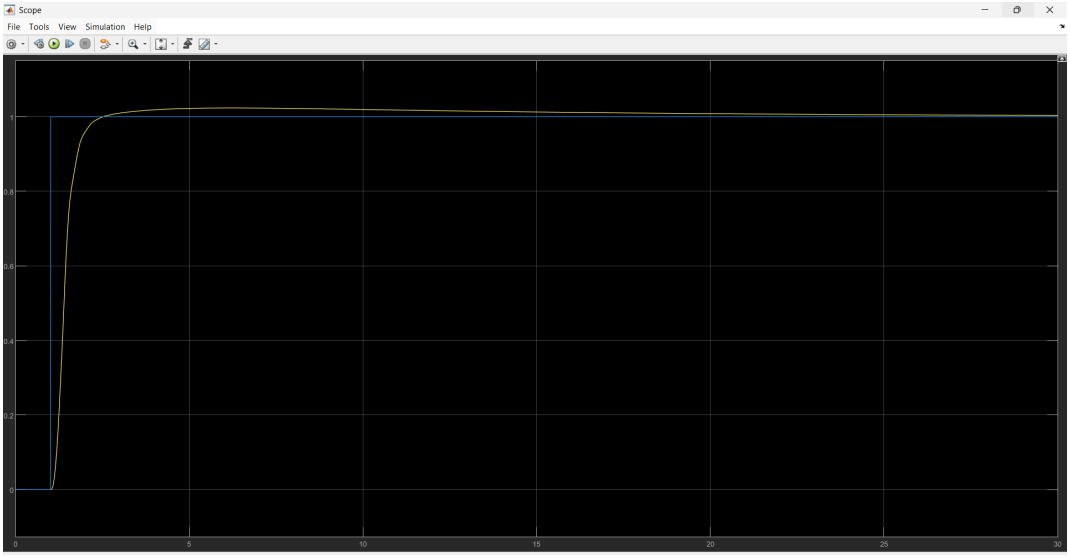


Figure 20: Closed-loop step response for Part (d).

Part (e)

The final plant was implemented utilizing a `Zero-Pole` block configured with the respective zeros, poles, and gain for the system.

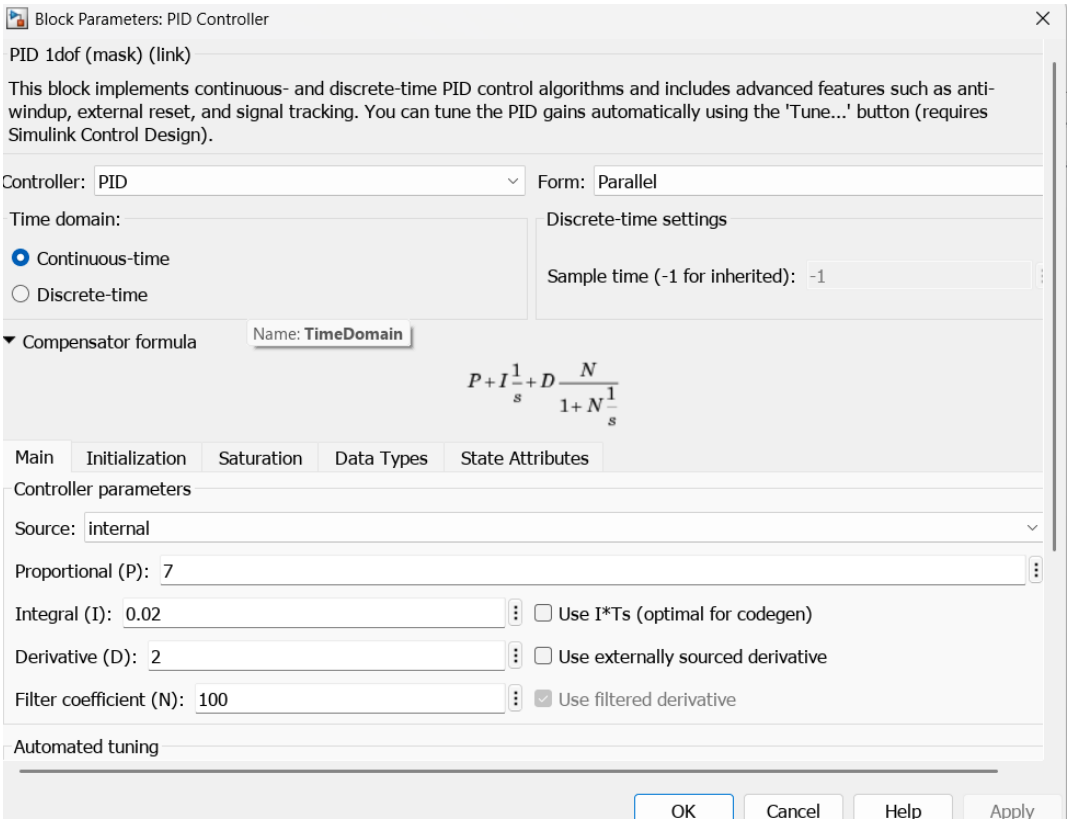


Figure 21: Calculated PID coefficients for Part (e).



Figure 22: Closed-loop step response for Part (e).

Q11 :

To design the PID controller for the system given in part (a) of Question 10, we strictly followed the standard PID Tuner App methodology.

First, we defined the plant transfer function $G_1(s)$ directly in the MATLAB Command Window so that it could be easily imported into the tuner. The following code was executed:

```
s = tf('s');
G1 = 1 / ( (s + 2) * (0.15*s + 1) * (0.05*s + 1) * (0.01*s + 1) );
```

```
> In pidtool (line 15)
In pidTuner (line 73)
>> s = tf('s');
G1 = 1 / ( (s + 2) * (0.15*s + 1) * (0.05*s + 1) * (0.01*s + 1) );
>>
>>
```

With the model generated in the workspace, we proceeded with the tuning process exactly by the book:

- **Steps I & II (Open & Import):** We launched the app using the `pidTuner` command and imported our `G1` plant model from the workspace.
- **Steps III & IV (Select & Initial Tune):** We selected the PID controller type and allowed the app to generate an initial baseline tuning based on its internal heuristics.
- **Steps V, VI & VII (Adjust, Measure & Iterate):** Using the interactive interface, we iteratively adjusted the response time and transient behavior. We continuously monitored performance metrics (rise time, settling time, overshoot) to minimize them and achieve the desired system response, while ensuring the input signal $u(t)$ remained in the feasible domain.
- **Steps VIII & IX (Validate & Export):** After achieving a satisfactory closed-loop response, we validated the system performance and exported the final controller parameters for implementation.

The tuning process successfully yielded an optimized controller. The step response demonstrating the system's performance, along with the final tuned parametric coefficients, are shown in the figures below.

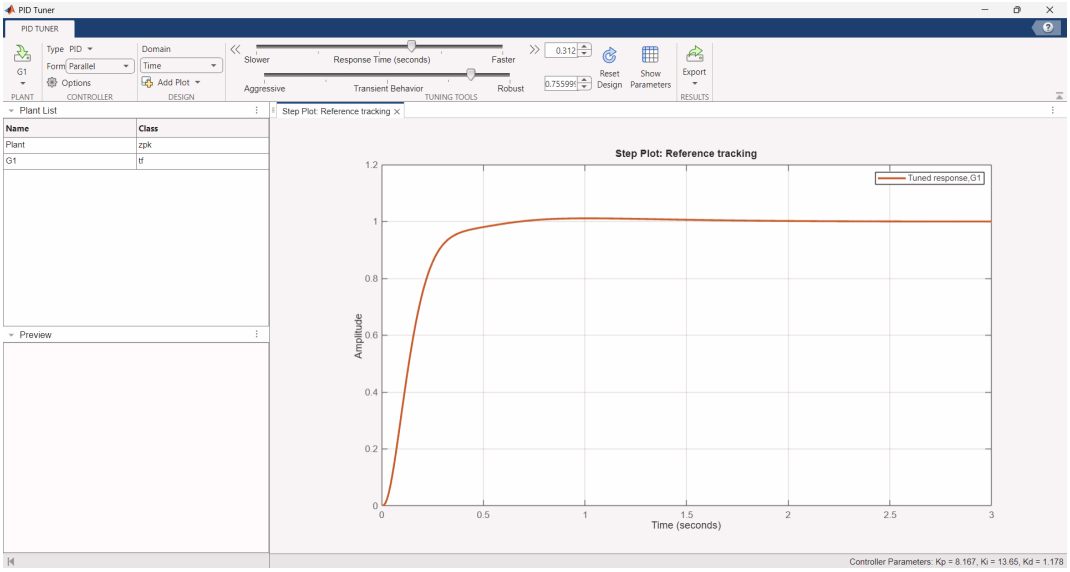


Figure 23: Closed-loop step response of the tuned system for $G_1(s)$.

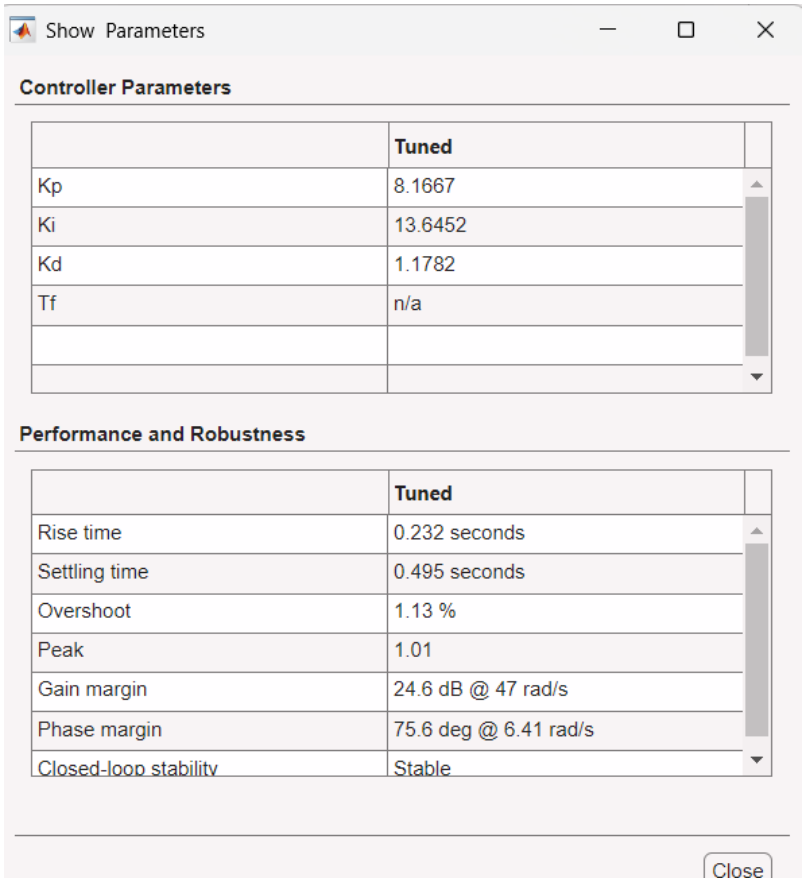


Figure 24: Tuned PID controller parameters and performance metrics.